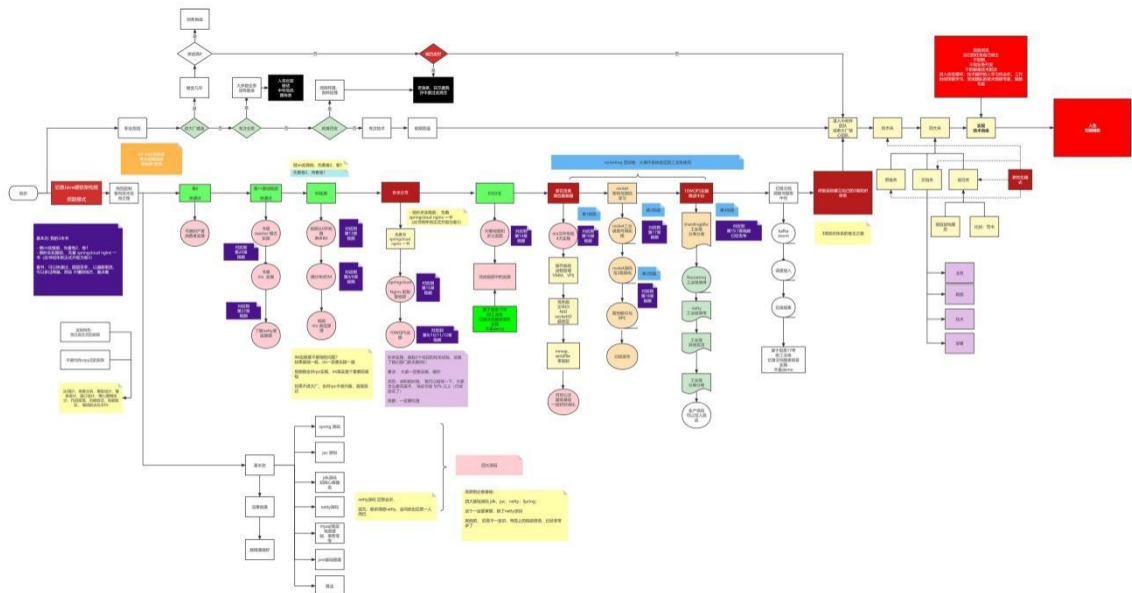


牛逼的职业发展之路

40 岁老架构尼恩用一张图揭秘：Java 工程师的高端职业发展路径，走向食物链顶端的之路

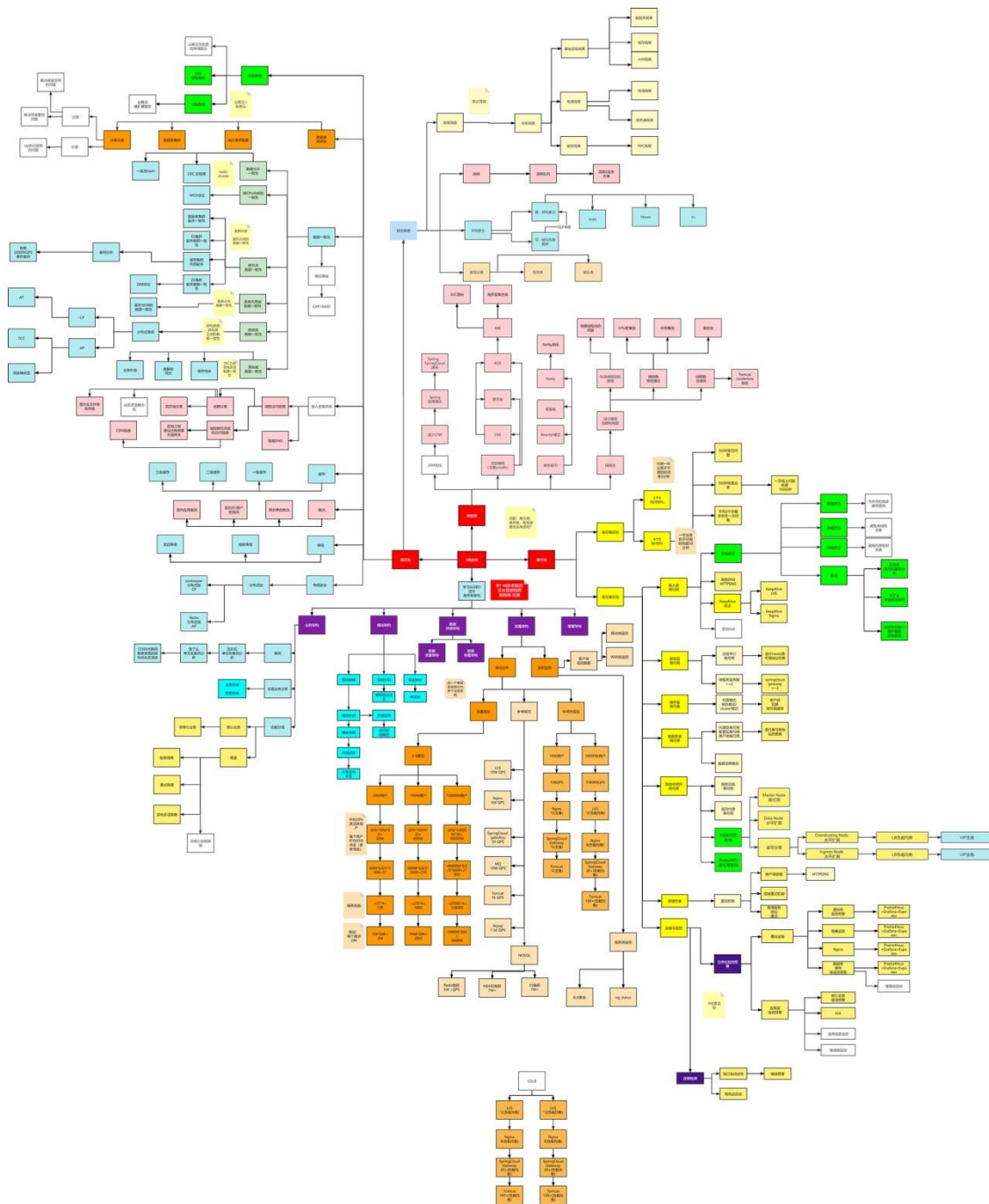
链接：<https://www.processon.com/view/link/618a2b62e0b34d73f7eb3cd7>



史上最全：价值10W的架构师知识图谱

此图梳理于尼恩的多个 3 高生产项目：多个亿级人民币的大型 SAAS 平台和智慧城市项目

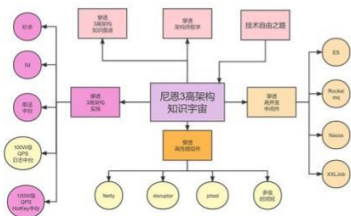
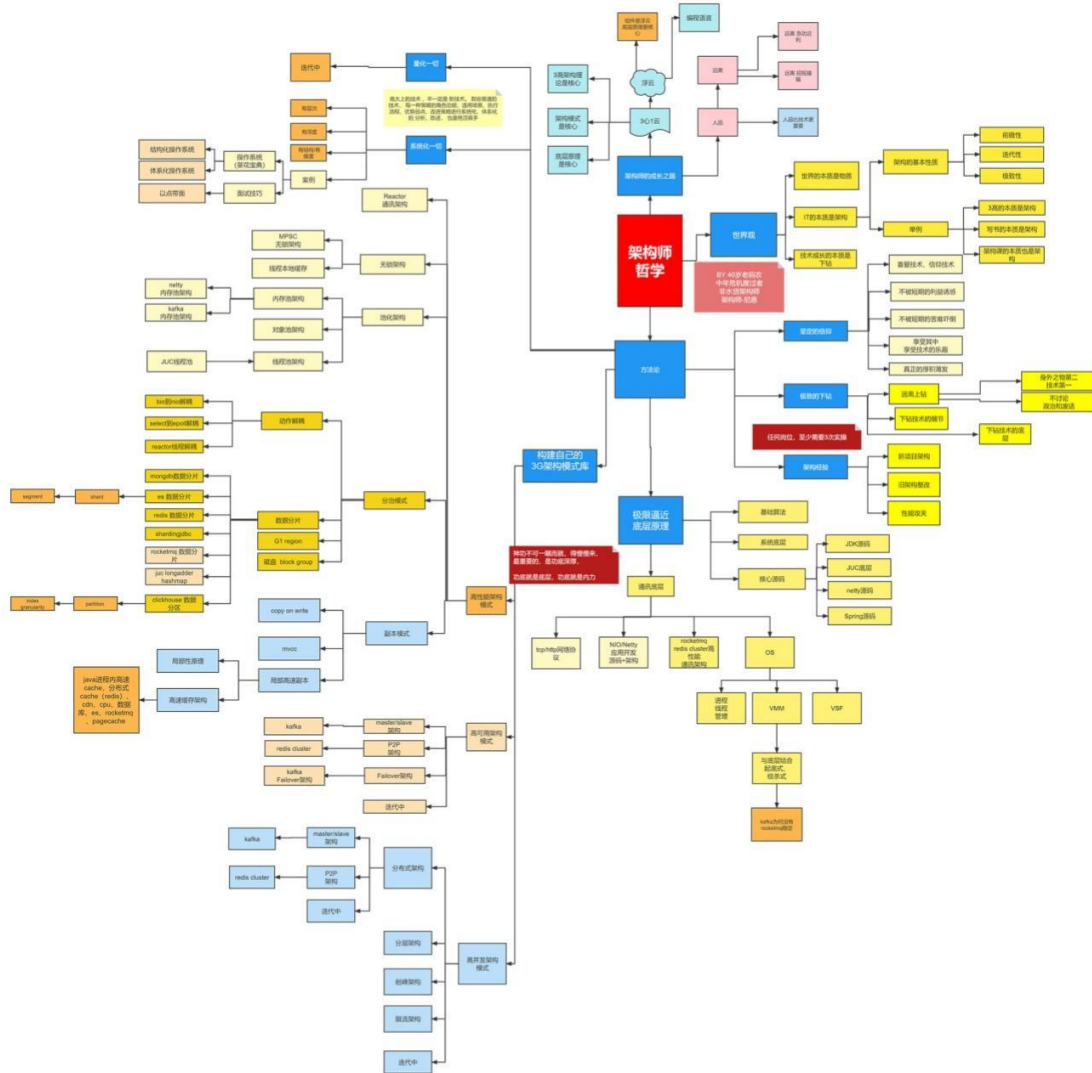
链接：<https://www.processon.com/view/link/60fb9421637689719d246739>



牛逼的架构师哲学

40 岁老架构师尼恩对自己的 20 年的开发、架构经验总结

链接: <https://www.processon.com/view/link/616f801963768961e9d9aec8>



牛逼的3高架构知识宇宙

尼恩 3 高架构知识宇宙，帮助大家穿透 3 高架构，走向技术自由，远离中年危机

链接: <https://www.processon.com/view/link/635097d2e0b34d40be778ab4>



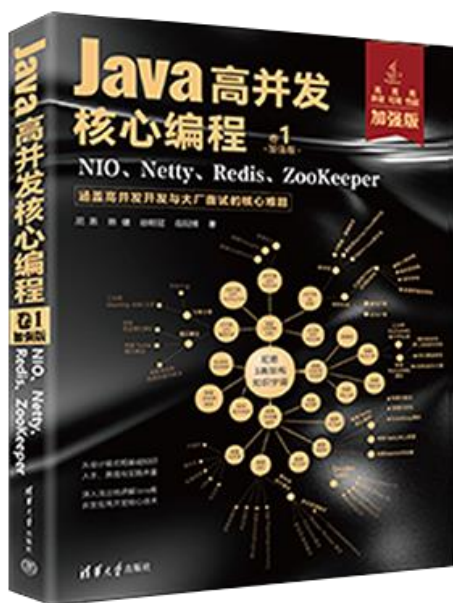
尼恩Java高并发三部曲（卷1加强版）

老版本：《Java 高并发核心编程 卷1：NIO、Netty、Redis、ZooKeeper》（已经过时，不建议购买）

新版本：《Java 高并发核心编程 卷1 **加强版**：NIO、Netty、Redis、ZooKeeper》

- 由浅入深地剖析了高并发 IO 的底层原理。
- 图文并茂地介绍了 TCP、HTTP、WebSocket 协议的核心原理。
- 细致深入地揭秘了 Reactor 高性能模式。
- 全面介绍了 Netty 框架，并完成单体 IM、分布式 IM 的实战设计。
- 详尽地介绍了 ZooKeeper、Redis 的使用，以帮助提升高并发、可扩展能力

详情：<https://www.cnblogs.com/crazymakercircle/p/16868827.html>



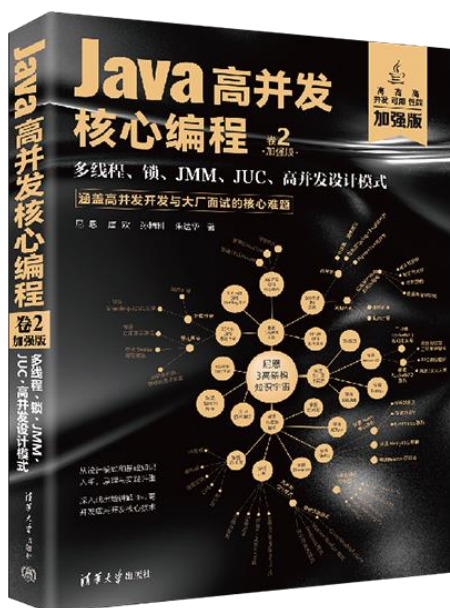
尼恩Java高并发三部曲（卷2加强版）

老版本：《Java 高并发核心编程 卷2：多线程、锁、JMM、JUC、高并发设计模式》
（已经过时，不建议购买）

新版本：《Java 高并发核心编程 卷2 **加强版**：多线程、锁、JMM、JUC、高并发设计模式》

- 由浅入深地剖析了 Java 多线程、线程池的底层原理。
- 总结了 IO 密集型、CPU 密集型线程池的线程数预估算法。
- 图文并茂的介绍了 Java 内置锁、JUC 显式锁的核心原理。
- 细致深入地揭秘了 JMM 内存模型。
- 全面介绍了 JUC 框架的设计模式与核心原理，并完成其高核心组件的实战介绍。
- 详尽地介绍了高并发设计模式的使用，以帮助提升高并发、可扩展能力

详情参阅：<https://www.cnblogs.com/crazymakercircle/p/16868827.html>



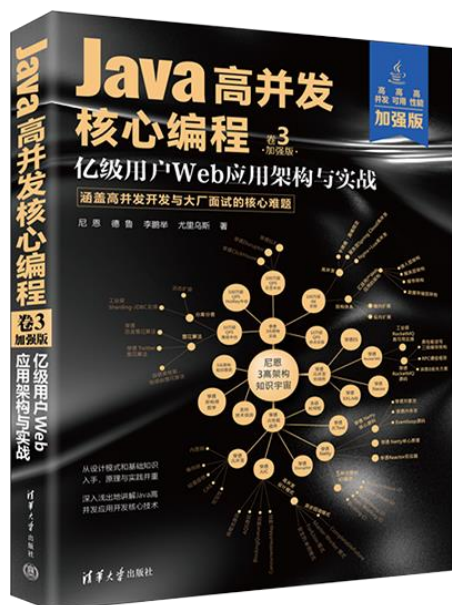
尼恩Java高并发三部曲（卷3加强版）

老版本：《SpringCloud Nginx 高并发核心编程》（已经过时，不建议购买）

新版本：《Java 高并发核心编程 卷3 **加强版**：亿级用户 Web 应用架构与实战》

- 在当今的面试场景中，3 高知识是大家面试必备的核心知识，本书基于亿级用户 3 高 Web 应用的架构分析理论，为大家对 3 高架构系统做一个系统化和清晰化的介绍。
- 从 Java 静态代理、动态代理模式入手，抽丝剥茧地解读了 SpringCloud 全家桶中 RPC 核心原理和执行过程，这是高级 Java 工程师面试必备的基础知识。
- 从Reactor 反应器模式入手，抽丝剥茧地解读了Nginx 核心思想和各配置项的底层知识和原理，这是高级 Java 工程师、架构师面试必备的基础知识。
- 从观察者模式入手，抽丝剥茧地解读了 RxJava、Hystrix 的核心思想和使用方法，这也是高级 Java 工程师、架构师面试必备的基础知识。

详情：<https://www.cnblogs.com/crazymakercircle/p/16868827.html>



尼恩Java面试宝典

35 个专题（卷王专供+ 史上最全 + 2023 面试必备）

详情：<https://www.cnblogs.com/crazymakercircle/p/13917138.html>



名称 专题01: JVM面试题 (卷王专供 + 史上最全 + 2023面试必备) -V10-from-尼恩Java面试宝典-release.pdf 专题02: Java算法面试题 (卷王专供 + 史上最全 + 2023面试必备) -V2 -from-Java面试红宝书-release.pdf 专题03: Java基础面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书 -release.pdf 专题04: 架构设计面试题 (卷王专供+ 史上最全 + 2023面试必备) -V10-from-Java面试红宝书-release.pdf 专题05: Spring面试题_专题06: SpringMVC_专题07: Tomcat面试题 (卷王专供+ 史上最全 + 2023面试必备) -V3-from-尼恩面试宝典-release.pdf 专题08: SpringBoot面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题09: 网络协议面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题10: TCP/IP协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题11: JUC并发包与容器类 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题12: 设计模式面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题13: 死锁面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题14: Redis 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V5-from-Java面试红宝书-release.pdf 专题15: 分布式锁 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题16: Zookeeper 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题17: 分布式事务面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题18: 一致性协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题19: Zab协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题20: Paxos 协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题21: raft 协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题22: Linux面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题23: Mysql 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V11-from-Java面试红宝书-release.pdf 专题24: SpringCloud 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V12-from-Java面试红宝书-release.pdf 专题25: Netty 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题26: 消息队列面试题: RabbitMQ、Kafka、RocketMQ (卷王专供+ 史上最全 + 2023面试必备) -V10-from-Java面试红宝书-release.pdf 专题27: 内存泄漏 内存溢出 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题28: JVM 内存溢出 实战 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题29: 多线程面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题30: HR面试题: 过五关斩六将后, 小心阴沟翻船! (史上最全、避坑宝典) -V2-from-Java面试红宝书-release.pdf 专题31: Hash链表面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题32: 大厂面试的基本流程和面试准备 (阿里、腾讯、网易、京东、头条.....) -V2-from-Java面试红宝书-release.pdf 专题33: BST、AVL、RBT红黑树、三大核心数据结构 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf 专题34: Elasticsearch面试题 (卷王专供+ 史上最全 + 2023面试必备) -V3-from-Java面试红宝书-release.pdf 专题35: Mybatis面试题 (卷王专供+ 史上最全 + 2023面试必备) -V3-from-尼恩Java面试宝典-release.pdf	
---	--

高性能核心组件之5：JCTool中 Mpsc 架构、源码分析

在大规模流量的高并发系统中，需要一些比链表结构性能更好的，基于数组 + CAS 操作实现的无锁安全队列，在行业中被广受认可的王者队列，是以下两个高性能无锁队列：

- Disruptor 环形队列
- JCTools MSPC 队列

Disruptor 环形队列已经在老的文章中，进行过非常详细的介绍。接下来，从架构、源码的角度，介绍一下 JCTools MSPC 队列。

这里解释一下 MSPC 的含义，如下：

- M: Multiple, 多个的。
- S: Single, 单个的。
- P: Producer, 生产者。
- C: Consumer, 消费者。

因此MpscQueue适用于多生产者，单消费者的高性能无锁队列。多生产者单消费者的使用场景，最佳的案例有：

- Netty Reactor 线程中任务队列 taskQueue 必须满足多个生产者可以同时提交任务。
- Caffeine中多个生产者业务线程，去进行缓存写入操作，而只有单个的维护线程，基于W-TinyLRU进行访问记录的维护

所以 JCTools 提供的 Mpsc Queue 非常适合：

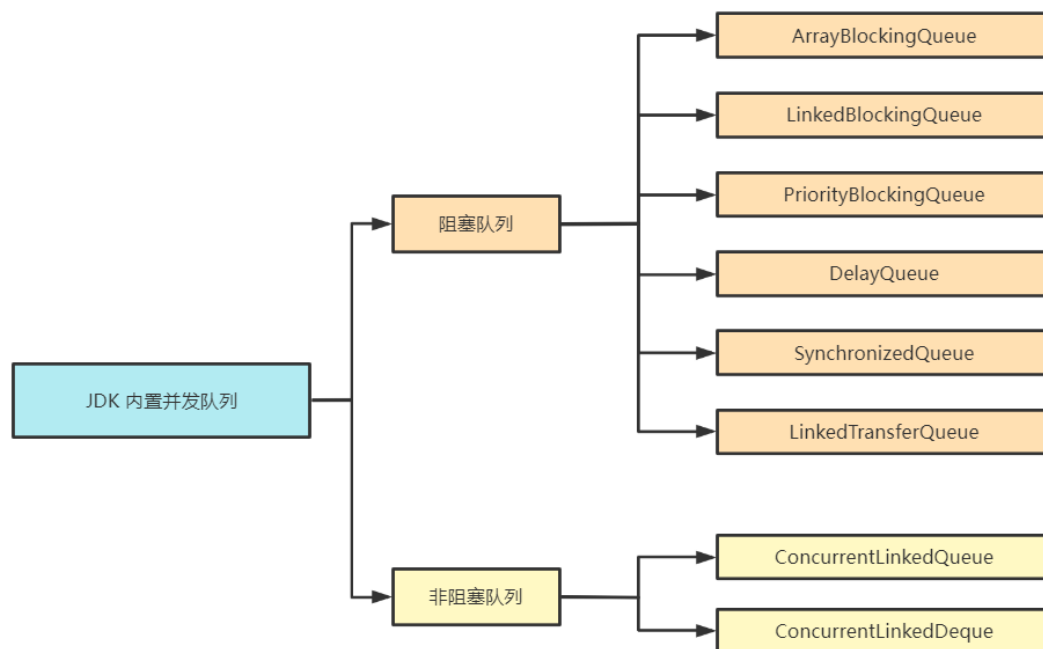
- Netty 异步任务场景，
- Caffeine的写入操作访问记录维护场景。

在介绍 Mpsc Queue 之前，我们先回顾下 JDK 原生队列的工作原理。

JDK 内置并发队列

JDK 内置并发队列按照实现方式可以分为阻塞队列和非阻塞队列两种类型，阻塞队列是基于锁实现的，非阻塞队列是基于 CAS 操作实现的。

JDK 中包含多种阻塞和非阻塞的队列实现，如下图所示。



CSDN @40岁资深老架构师尼恩

队列是一种 FIFO（先进先出）的数据结构，JDK 中定义了 `java.util.Queue` 的队列接口，与 `List`、`Set` 接口类似，`java.util.Queue` 也继承于 `Collection` 集合接口。

此外，JDK 还提供了一种双端队列接口 `java.util.Deque`，我们最常用的 `LinkedList` 就是实现了 `Deque` 接口。

下面我们简单说说上图中的每个队列的特点，并给出一些对比和总结。

阻塞队列

阻塞队列在队列为空或者队列满时，都会发生阻塞。阻塞队列自身是线程安全的，使用者无需关心线程安全问题，降低了多线程开发难度。

阻塞队列主要分为以下几种：

- **ArrayBlockingQueue：**

最基础且开发中最常用的阻塞队列，底层采用数组实现的有界队列，初始化需要指定队列的容量。`ArrayBlockingQueue` 是如何保证线程安全的呢？

它内部是使用了一个重入锁 `ReentrantLock`，并搭配 `notEmpty`、`notFull` 两个条件变量 `Condition` 来控制并发访问。

从队列读取数据时，如果队列为空，那么会阻塞等待，直到队列有数据了才会被唤醒。

如果队列已经满了，也同样会进入阻塞状态，直到队列有空闲才会被唤醒。

- **LinkedBlockingQueue：**

内部采用的数据结构是链表，队列的长度可以有界或者无界的，初始化不需要指定队列长度，默认是 `Integer.MAX_VALUE`。

`LinkedBlockingQueue` 内部使用了 `takeLock`、`putLock` 两个重入锁 `ReentrantLock`，以及 `notEmpty`、`notFull` 两个条件变量 `Condition` 来控制并发访问。

采用读锁和写锁的好处是可以避免读写时相互竞争锁的现象，所以相比于 `ArrayBlockingQueue`，`LinkedBlockingQueue` 的性能要更好。

- **PriorityBlockingQueue：**

采用最小堆实现的优先级队列，队列中的元素按照优先级进行排列，每次出队都是返回优先级最高的元素。PriorityBlockingQueue 内部是使用了一个 ReentrantLock 以及一个条件变量 Condition notEmpty 来控制并发访问，

因为 PriorityBlockingQueue 是无界队列，所以不需要 notFull，每次 put 都不会发生阻塞。PriorityBlockingQueue 底层的最小堆是采用数组实现的，当元素个数大于等于最大容量时会触发扩容，

在扩容时会先释放锁，保证其他元素可以正常出队，然后使用 CAS 操作确保只有一个线程可以执行扩容逻辑。

- **DelayQueue,**

一种支持延迟获取元素的阻塞队列，常用于缓存、定时任务调度等场景。

DelayQueue 内部是采用优先级队列 PriorityQueue 存储对象。

DelayQueue 中的每个对象都必须实现 Delayed 接口，并重写 compareTo 和 getDelay 方法。向队列中存放元素的时候必须指定延迟时间，只有延迟时间已满的元素才能从队列中取出。

- **SynchronizedQueue,**

又称无缓冲队列。

比较特别的是 SynchronizedQueue 内部不会存储元素。与 ArrayBlockingQueue、LinkedBlockingQueue 不同，SynchronizedQueue 直接使用 CAS 操作控制线程的安全访问。

其中 put 和 take 操作都是阻塞的，每一个 put 操作都必须阻塞等待一个 take 操作，反之亦然。

所以 SynchronizedQueue 可以理解为生产者和消费者配对的场景，双方必须互相等待，直至配对成功。

在 JDK 的线程池 Executors.newCachedThreadPool 中就存在 SynchronousQueue 的运用，对于新提交的任务，如果有空闲线程，将重复利用空闲线程处理任务，否则将新建线程进行处理。

- **LinkedTransferQueue**

一种特殊的无界阻塞队列，可以看作 LinkedBlockingQueues、SynchronousQueue（公平模式）、ConcurrentLinkedQueue 的合体。

与 SynchronousQueue 不同的是，LinkedTransferQueue 内部可以存储实际的数据，当执行 put 操作时，如果有等待线程，那么直接将数据交给对方，否则放入队列中。与 LinkedBlockingQueues 相比，LinkedTransferQueue 使用 CAS 无锁操作进一步提升了性能。

非阻塞队列

说完阻塞队列，我们再来看下非阻塞队列。

非阻塞队列不需要通过加锁的方式对线程阻塞，并发性能更好。

JDK 中常用的非阻塞队列有以下几种：

- **ConcurrentLinkedQueue,**

它是一个采用双向链表实现的无界并发非阻塞队列，它属于 LinkedQueue 的安全版本。ConcurrentLinkedQueue 内部采用 CAS 操作保证线程安全，这是非阻塞队列实现的基础，相比 ArrayBlockingQueue、LinkedBlockingQueue 具备较高的性能。

- **ConcurrentLinkedDeque,**

也是一种采用双向链表结构的无界并发非阻塞队列。

与 ConcurrentLinkedQueue 不同的是，ConcurrentLinkedDeque 属于双端队列，

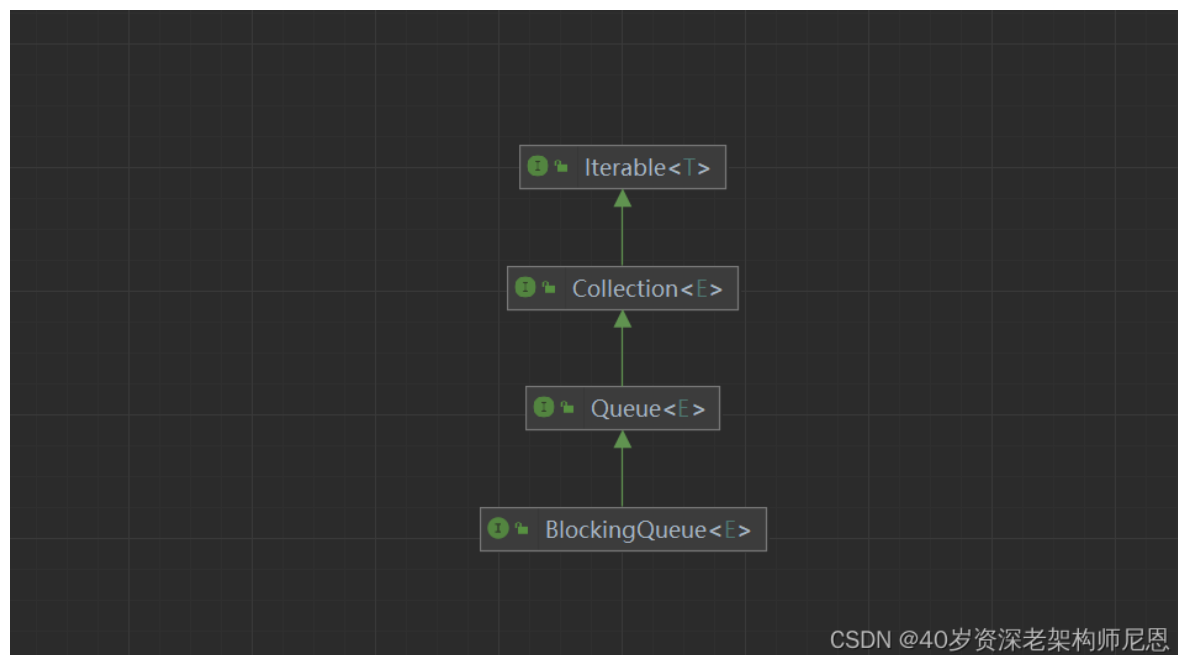
它同时支持 FIFO 和 FILO 两种模式，可以从队列的头部插入和删除数据，也可以从队列尾部插入和删除数据，适用于多生产者和多消费者的场景。

BlockingQueue阻塞队列超级接口

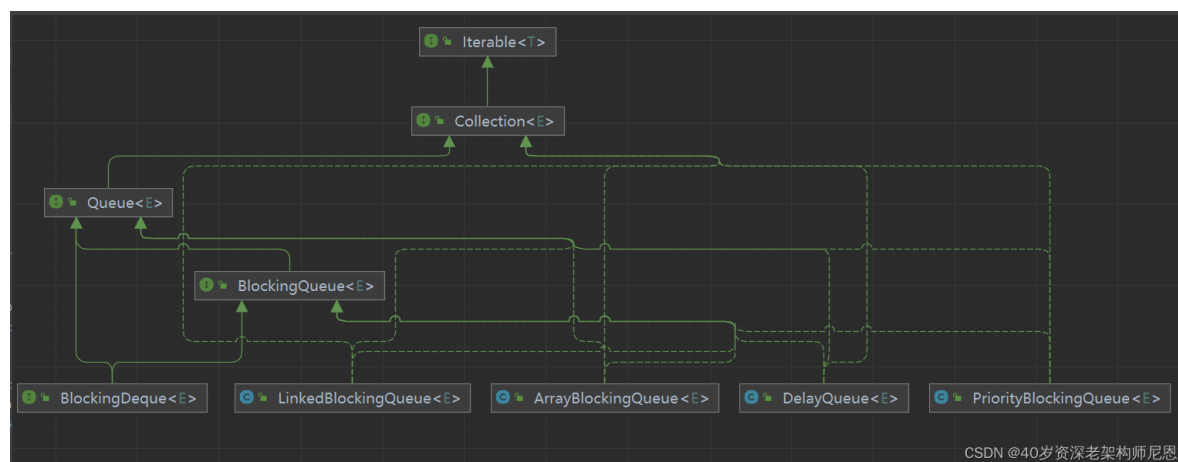
至此，常见的队列类型我们已经介绍完了。我们在平时开发中使用频率最高的是 BlockingQueue。

实现一个阻塞队列需要具备哪些基本功能呢？

下面看 BlockingQueue 的接口继承关系，如下图所示。



下面看 BlockingQueue 的接口的各种实现子类，如下图所示。



BlockingQueue 实现子类太多，图里放不下，还请大家通过IDEA工具，自行查看。

下面看 BlockingQueue 的接口的抽象方法，如下图所示。

BlockingDeque<E>

		<code>remove(Object)</code>	<code>boolean</code>
		<code>push(E)</code>	<code>void</code>
		<code>size()</code>	<code>int</code>
		<code>offer(E)</code>	<code>boolean</code>
		<code>offerLast(E, long, TimeUnit)</code>	<code>boolean</code>
		<code>addLast(E)</code>	<code>void</code>
		<code>pollFirst(long, TimeUnit)</code>	<code>E?</code>
		<code>removeLastOccurrence (Object)</code>	<code>boolean</code>
		<code>offer(E, long, TimeUnit)</code>	<code>boolean</code>
		<code>offerFirst(E)</code>	<code>boolean</code>
		<code>putFirst(E)</code>	<code>void</code>
		<code>add(E)</code>	<code>boolean</code>
		<code>element()</code>	<code>E</code>
		<code>iterator()</code>	<code>Iterator <E></code>
		<code>put(E)</code>	<code>void</code>
		<code>removeFirstOccurrence (Object)</code>	<code>boolean</code>
		<code>peek()</code>	<code>E</code>
		<code>pollLast(long, TimeUnit)</code>	<code>E?</code>
		<code>takeLast()</code>	<code>E</code>
		<code>addFirst(E)</code>	<code>void</code>
		<code>takeFirst()</code>	<code>E</code>
		<code>remove()</code>	<code>E</code>
		<code>contains(Object)</code>	<code>boolean</code>
		<code>putLast(E)</code>	<code>void</code>
		<code>take()</code>	<code>E</code>
		<code>offerLast(E)</code>	<code>boolean</code>
		<code>poll(long, TimeUnit)</code>	<code>E?</code>
		<code>poll()</code>	<code>E</code>
		<code>offerFirst(E, long, TimeUnit)</code>	<code>boolean</code>

CSDN @40岁资深老架构师尼恩

我们可以通过下面一张表格，对上述 BlockingQueue 接口的具体行为进行归类。

以下表格，来自于《Java高并发核心编程 卷2 加强版》

3. 获取元素类方法

- 1) `element()`: 获取但不移除此队列的队首元素，没有元素则抛出异常。
- 2) `peek()`: 获取但不移除此队列的队首元素；若队列为空，则返回`null`。

阻塞队列对元素的增删查操作主要就是上述的三类方法，这里对这三类方法的特征进行总结，具体如表7-1所示。

表 7-1 阻塞队列三类方法的特征

	抛出异常	特 殊 值	阻 塞	限时阻塞
添 加	<code>add(e)</code>	<code>offer(e)</code>	<code>put(e)</code>	<code>offer(e, time, unit)</code>
删 除	<code>remove()</code>	<code>poll()</code>	<code>take()</code>	<code>poll(time, unit)</code>
获取元素	<code>element()</code>	<code>peek()</code>	不可用	不可用

对表7-1中的4个特征说明如下：

CSDN @40岁资深老架构师尼恩

Java内置队列的问题

我们先来看一看常用的线程安全的内置队列有什么问题。Java的内置队列如下表所示。

队列	有界性	锁	数据结构
<code>ArrayBlockingQueue</code>	<code>bounded</code>	加锁	<code>arraylist</code>
<code>LinkedBlockingQueue</code>	<code>optionally-bounded</code>	加锁	<code>linkedList</code>
<code>ConcurrentLinkedQueue</code>	<code>unbounded</code>	无锁	<code>linkedList</code>
<code>LinkedTransferQueue</code>	<code>unbounded</code>	无锁	<code>linkedList</code>
<code>PriorityBlockingQueue</code>	<code>unbounded</code>	加锁	<code>heap</code>
<code>DelayQueue</code>	<code>unbounded</code>	加锁	<code>heap</code>

队列的底层一般分成三种：数组、链表和堆。

其中，堆一般情况下是为了实现带有优先级特性的队列，暂时不做介绍，后面结合堆的结构，做专题介绍，这个也是一个 $O(\log n)$ 的性能比较高的结构。

从数组和链表两种数据结构来看，两类结构如下：

- 基于数组线程安全的队列，比较典型的是`ArrayBlockingQueue`，它主要通过加锁的方式来保证线程安全；
- 基于链表的线程安全队列分成`LinkedBlockingQueue`和`ConcurrentLinkedQueue`两大类，前者也通过锁的方式来实现线程安全，而后者是无锁结构，通过原子变量`compare and swap`（以下简称“CAS”）这种**无锁方式**来实现的。

LinkedTransferQueue和ConcurrentLinkedQueue一样，也是无锁结构，通过原子变量compare and swap（以下简称“CAS”）这种不加锁的方式来实现的，性能还是比较高的。

那么LinkedTransferQueue和ConcurrentLinkedQueue 的问题是啥呢？

链表结构的核心的问题是：

链接的节点，在高并发的场景下，存在者大量的节点创建/GC回收的压力，这会导致STW的产生，容易出现业务卡顿。

另外，LinkedTransferQueue 对和ConcurrentLinkedQueue 对 volatile类型的变量进行 CAS 操作，存在伪共享问题，

总之，在高并发的场景下，链表结构没有数组结构，性能那么优越。

而JDK内置的基于数据结构的ArrayBlockingQueue队列，有一个核心问题：

有锁结构，高并发场景，性能没有无锁结构，那么高

另外，最好是基于数组结构的环形队列模式，能最大的限度的复用内存空间，减少内存分配和GC的压力。

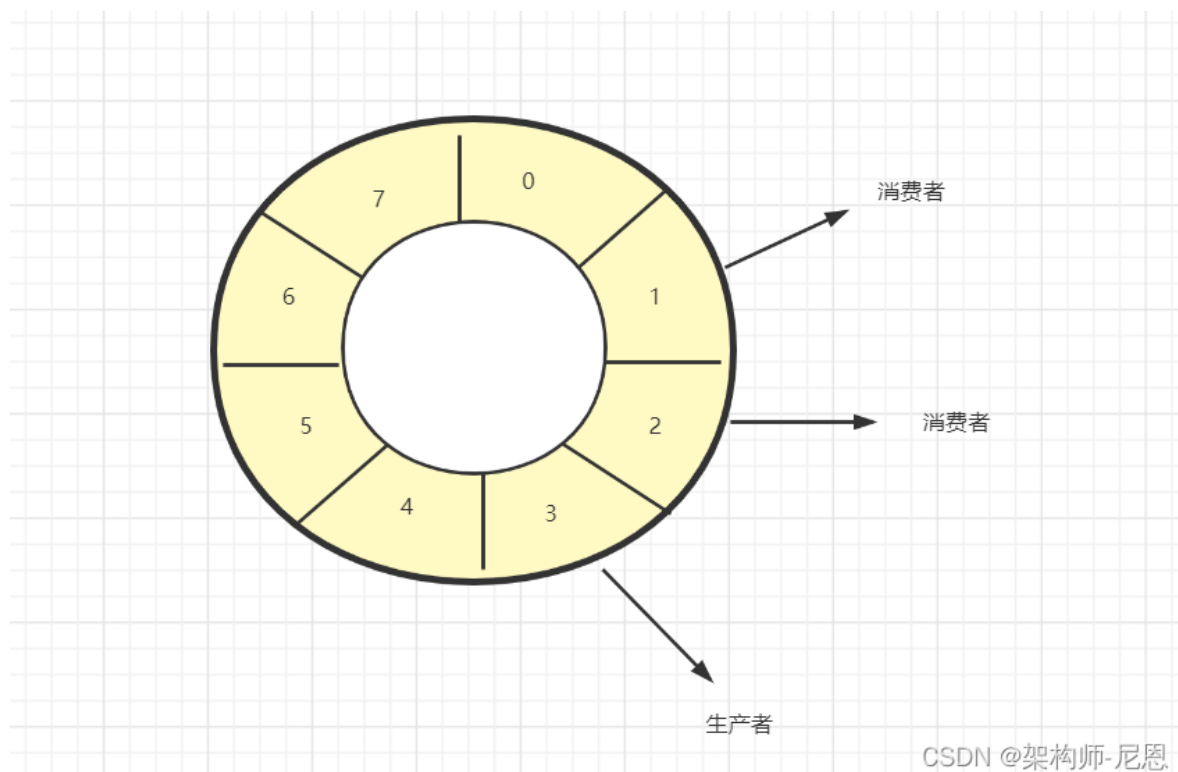
高性能的数组 + CAS无锁队列解决访问

在大规模流量的高并发系统中，需要一个数组 + CAS 操作实现的无锁安全队列，有没有成熟的解决方案呢？

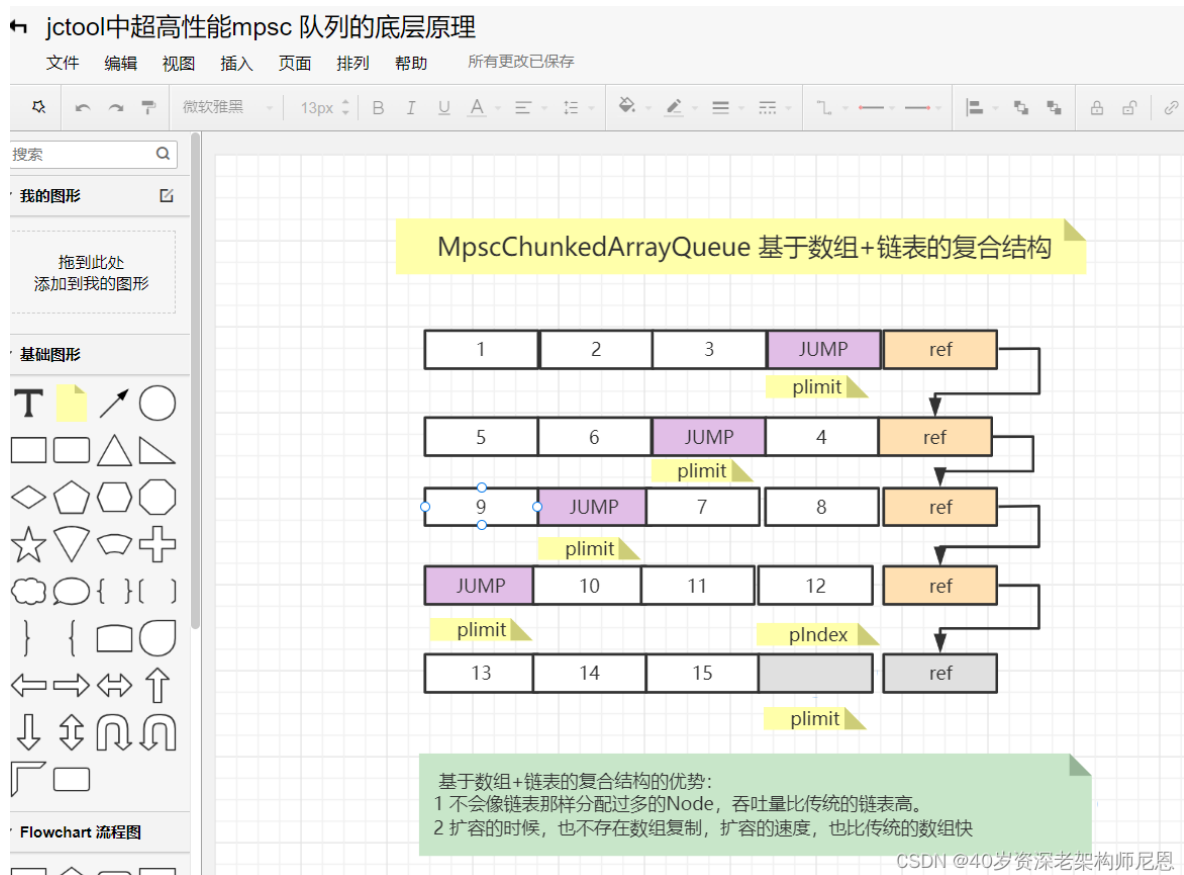
有两个高性能无锁队列：

- Disruptor
- JCTools

Disruptor 采用的是环形结构，进行无锁队列的设计



JCTools采用的是 数组+链表的复合结构，进行无锁队列的设计



Disruptor如何实现高性能？

使用Disruptor，主要用于对性能要求高、延迟低的场景，它通过“榨干”机器的性能来换取处理的高性能。

Disruptor实现高性能主要体现了去掉了锁，采用CAS算法，同时内部通过环形队列实现有界队列。

- 环形数据结构

数组元素不会被回收，避免频繁的GC，所以，为了避免垃圾回收，采用数组而非链表。

同时，数组对处理器的缓存机制更加友好。

预分配数组空间

- 无锁设计

采用CAS无锁方式，保证线程的安全性

每个生产者或者消费者线程，会先申请可以操作的元素在数组中的位置，申请到之后，直接在该位置写入或者读取数据。

整个过程通过原子变量CAS，保证操作的线程安全。

为啥cas性能高，请参见《java高并发核心编程卷2》

- 元素位置属性进行cacheline填充：

通过添加额外的无用信息，避免伪共享问题

性能提升5倍多

- 元素位置属性进行volatile变量延迟写入（有序写入）：

性能提升10倍多

- 位运算定位元素在数组当中的位置

数组长度 2^n ，通过位运算，加快定位的速度。

下标采取递增的形式。

不用担心index溢出的问题。index是long类型，即使100万QPS的处理速度，也需要30万年才能用完。

Disruptor 已经开源，详细可查阅 Github 地址 <https://github.com/LMAX-Exchange/disruptor>。

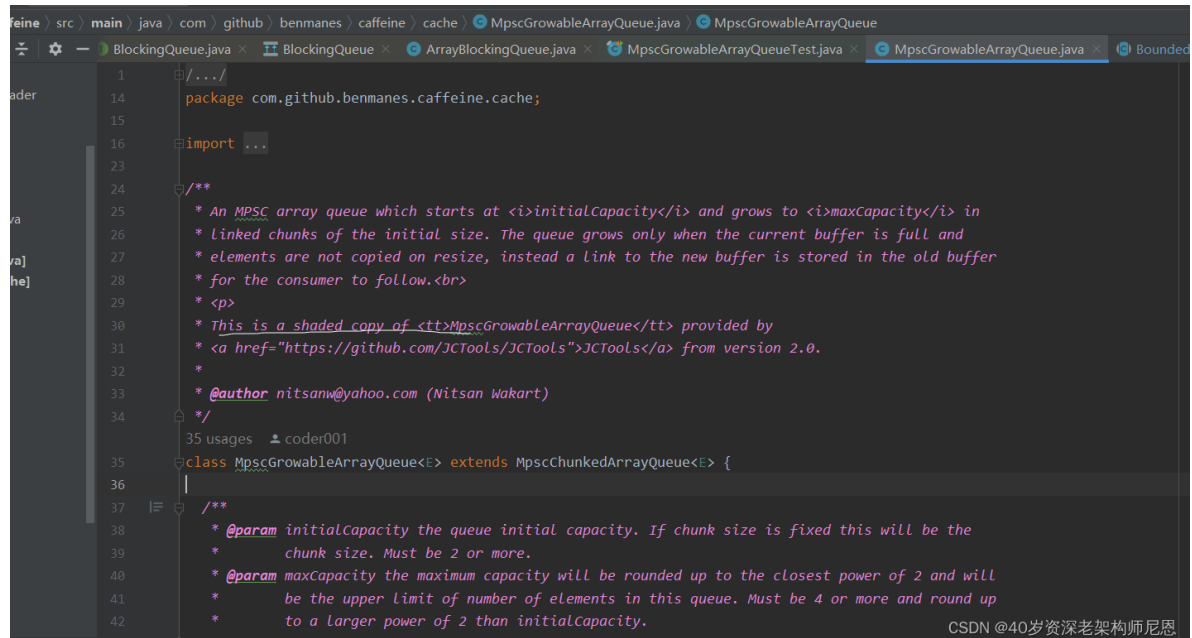
JCTools如何实现高性能？

JCTools 也是一个开源项目，Github 地址为 <https://github.com/JCTools/JCTools>。

JCTools 是适用于 JVM 并发开发的工具，主要提供了一些 JDK 缺失的并发数据结构，例如非阻塞 Map、非阻塞 Queue 等。其中非阻塞队列可以分为四种类型，可以根据不同的场景选择使用。

- Spsc 单生产者单消费者；
- Mpsc 多生产者单消费者；
- Spmc 单生产者多消费者；
- Mpmc 多生产者多消费者。

Netty 中直接引入了 JCTools 的 Mpsc Queue，Caffeine 中引入了 JCTools 的 Mpsc Queue，复制了其代码，然后简单改了下。



```
1  package com.github.benmanes.caffeine.cache;
14
15
16  import ...
17
18  /**
19   * An MPSC array queue which starts at <i>initialCapacity</i> and grows to <i>maxCapacity</i> in
20   * linked chunks of the initial size. The queue grows only when the current buffer is full and
21   * elements are not copied on resize, instead a link to the new buffer is stored in the old buffer
22   * for the consumer to follow.
23   *
24   * This is a shaded copy of <code>MpscGrowableArrayQueue</code> provided by
25   * <a href="https://github.com/JCTools/JCTools">JCTools</a> from version 2.0.
26   *
27   * @author nitsanw@yahoo.com (Nitsan Wakart)
28   */
29
30  35 usages  coder001
31
32  class MpscGrowableArrayQueue<E> extends MpscChunkedArrayQueue<E> {
33
34      /**
35       * @param initialCapacity the queue initial capacity. If chunk size is fixed this will be the
36       * chunk size. Must be 2 or more.
37       * @param maxCapacity the maximum capacity will be rounded up to the closest power of 2 and will
38       * be the upper limit of number of elements in this queue. Must be 4 or more and round up
39       * to a larger power of 2 than initialCapacity.
40       */
41  }
```

相比于 JDK 原生的并发队列，Mpsc Queue 又有什么过人之处呢？

Mpsc Queue 基础知识

Mpsc 的全称是 Multi Producer Single Consumer，多生产者单消费者。

Mpsc Queue 可以保证多个生产者同时访问队列是线程安全的，而且同一时刻只允许一个消费者从队列中读取数据。

多生产者单消费者的使用场景，最佳的案例有：

- Netty Reactor 线程中任务队列 taskQueue 必须满足多个生产者可以同时提交任务。
- Caffeine中多个生产者业务线程，去进行缓存写入操作，而只有单个的维护线程，基于W-TinyLRU进行访问记录的维护

所以JCTools 提供的 Mpsc Queue 非常适合：

- Netty 异步任务场景，
- Caffeine的写入操作访问记录维护场景.

Mpsc Queue 有多种的实现类，例如 MpscArrayQueue、MpscUnboundedArrayQueue、MpscChunkedArrayQueue 等。

首先我们看下 MpscArrayQueue 的继承关系



除了顶层 JDK 原生的 AbstractCollection、AbstractQueue接口之外，MpscArrayQueue 还继承了很多类似于 MpscXxxPad 以及 MpscXxxField 的类。

高性能组件的Cache Line Padding缓存行填充 技术

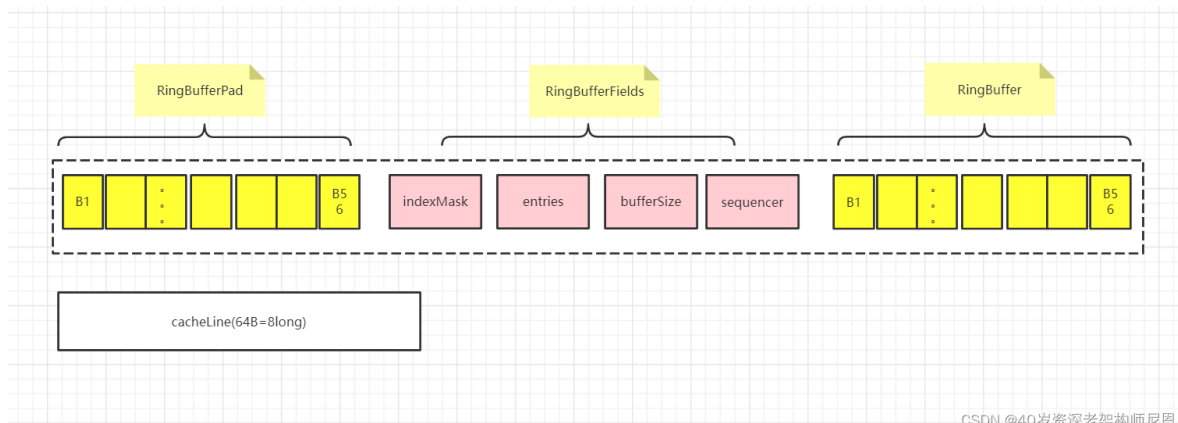
我们可以发现一个很有意思的规律，JCTool 每个有包含属性的类后面都会被 MpscXxxPad 类隔开。

MpscXxxPad 到底起到什么作用呢？

首先，回顾一下 Disruptor RingBuffer 的缓存行填充

Disruptor RingBuffer 的缓存行填充

Disruptor RingBuffer（环形缓冲区）定义了RingBufferFields类，里面有indexMask和其他几个变量存放RingBuffer的内部状态信息。

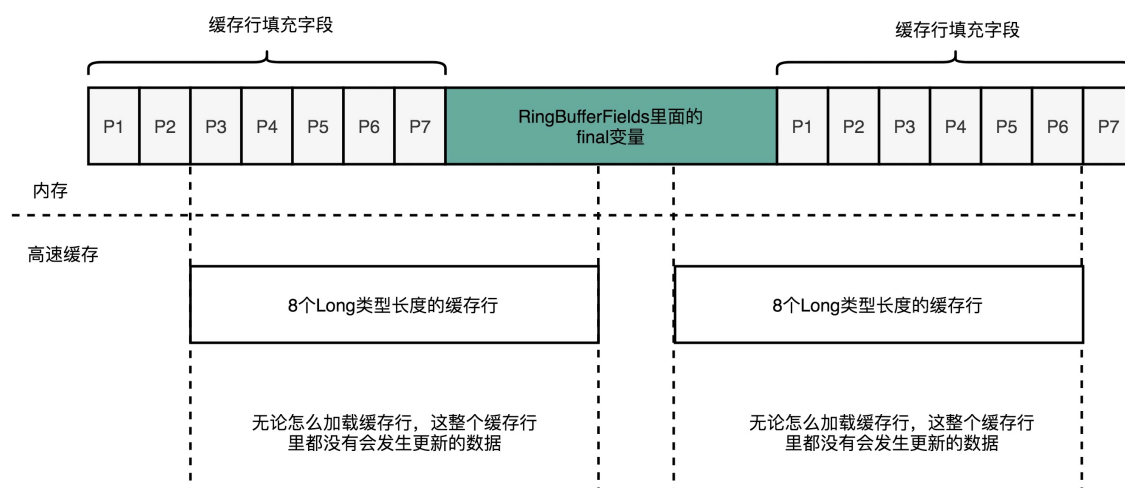


Disruptor利用了缓存行填充，在 RingBufferFields里面定义的变量的前后，分别定义了7个long类型的变量：

- 前面7个来自继承的 RingBufferPad 类
- 后面7个直接定义在 RingBuffer 类

这14个变量无任何实际用途。我们既不读他们，也不写他们。帮助RingBufferFields 进行缓存行填充 Cache Line Padding

所以，一旦 RingBufferFields 被加载到CPU Cache后，只要被频繁读取访问，就不会再被换出Cache。这意味着，对于RingBufferFields 的读取速度，会一直是CPU Cache的访问速度，而非内存的访问速度。



MPSC的RingBuffer 的缓存行填充

我们自顶向下，将所有类的字段合并在一起，看下 MpscArrayQueue 的整体结构。

除了顶层JDK 原生的 AbstractCollection、AbstractQueue, MpscArrayQueue 还继承了很多类似于 MpscXxxPad 以及 MpscXxxField 的类。

我们可以发现一个很有意思的规律：

每个有包含属性的类后面都会被 MpscXxxPad 类隔开。

MpscXxxPad 到底起到什么作用呢？

我们自顶向下，将所有类的字段合并在一起，看下 MpscArrayQueue 的整体结构。

- 填充类 BaseMpscLinkedArrayQueuePad1

```
abstract class BaseMpscLinkedArrayQueuePad1<E> extends AbstractQueue<E>
implements IndexedQueue {
    long p01, p02, p03, p04, p05, p06, p07;
    long p10, p11, p12, p13, p14, p15, p16, p17;
}
```

- 包含属性的类 BaseMpscLinkedArrayQueueProducerFields
包含生产相关的属性

```
// $gen:ordered-fields
abstract class BaseMpscLinkedArrayQueueProducerFields<E> extends
BaseMpscLinkedArrayQueuePad1<E> {
    private final static long P_INDEX_OFFSET =
fieldOffset(BaseMpscLinkedArrayQueueProducerFields.class, "producerIndex");

    private volatile long producerIndex;

    // 获取生产者索引
    // lv = lazy value
    @Override
    public final long lvProducerIndex() {
        return producerIndex;
    }

    // so = set ordered / lazy set
    final void soProducerIndex(long newValue) {
        UNSAFE.putOrderedLong(this, P_INDEX_OFFSET, newValue);
    }

    final boolean casProducerIndex(long expect, long newValue) {
        return UNSAFE.compareAndSwapLong(this, P_INDEX_OFFSET, expect,
newValue);
    }
}
```

- 填充类 BaseMpscLinkedArrayQueuePad2

```

abstract class BaseMpscLinkedListArrayQueuePad2<E> extends
BaseMpscLinkedListArrayQueueProducerFields<E> {
    long p01, p02, p03, p04, p05, p06, p07;
    long p10, p11, p12, p13, p14, p15, p16, p17;
}

```

- 包含属性的类 BaseMpscLinkedListArrayQueueConsumerFields
包含消费者相关的属性

```

// $gen:ordered-fields
abstract class BaseMpscLinkedListArrayQueueConsumerFields<E> extends
BaseMpscLinkedListArrayQueuePad2<E> {
    private final static long C_INDEX_OFFSET =
fieldOffset(BaseMpscLinkedListArrayQueueConsumerFields.class, "consumerIndex");

    private volatile long consumerIndex;
    protected long consumerMask;
    protected E[] consumerBuffer;

    // load volatile 消费者索引
    @Override
    public final long lvConsumerIndex() {
        return consumerIndex;
    }

    // load pain
    final long lpConsumerIndex() {
        return UNSAFE.getLong(this, C_INDEX_OFFSET);
    }

    // store ordered 消费者索引
    final void soConsumerIndex(long newValue) {
        UNSAFE.putOrderedLong(this, C_INDEX_OFFSET, newValue);
    }
}

```

可以看出，MpscXxxPad 类中使用了大量 long 类型的变量，其命名没有什么特殊的含义，只是起到填充的作用。

和Disruptor 的源码，Mpsc Queue 和 Disruptor 之所以填充这些无意义的变量，是为了解决伪共享（false sharing）问题。

什么是伪共享呢？

请参考第 23 章：100w 级别 qps 日志平台，

那里介绍的极致详细，还有性能对比实操测试。已经是史诗级的详细了，这里不做介绍。

MpscGrowableArrayQueue 源码分析

MpscArrayQueue 基本用法与 ArrayBlockingQueue 都是类似的：入队 offer()和出队 poll()。

MpscArrayQueue 如何使用，示例代码如下：

```
package org.jctools.demo;

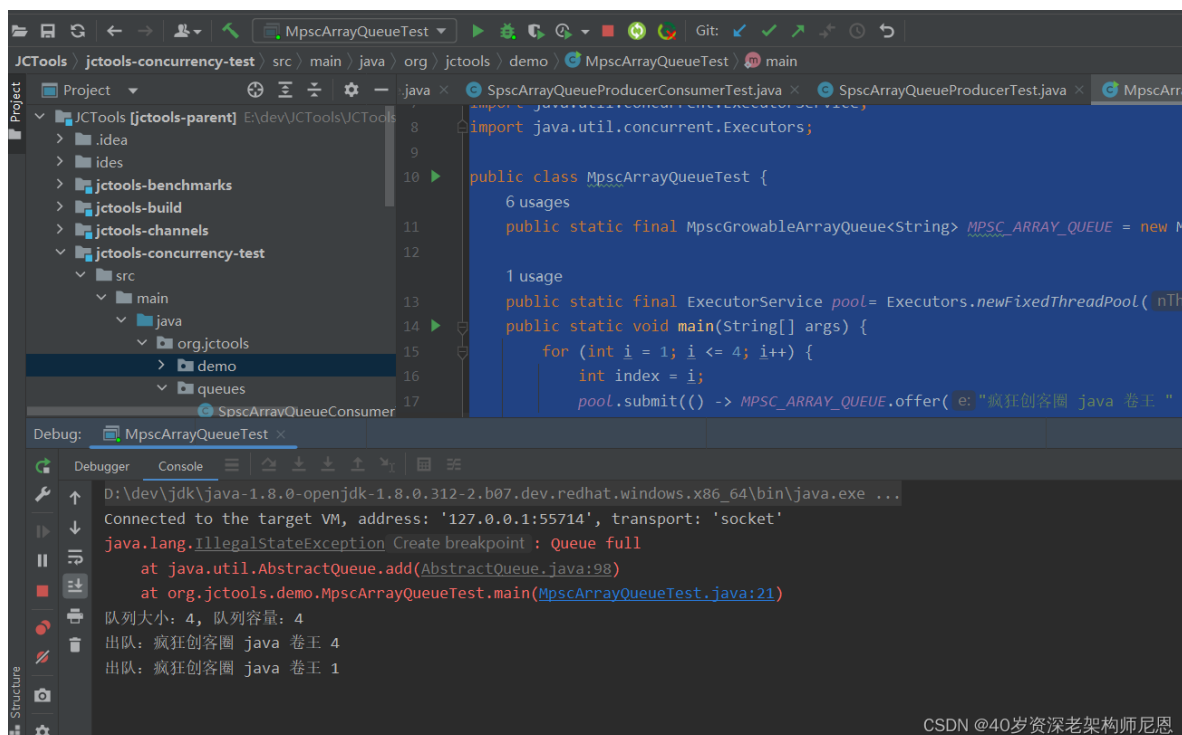
import org.jctools.queues.MpscArrayQueue;
import org.jctools.queues.MpscGrowableArrayQueue;

import java.util.concurrent.Executor;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class MpscArrayQueueTest {
    public static final MpscGrowableArrayQueue<String> MPSC_ARRAY_QUEUE = new
MpscGrowableArrayQueue<>(4);

    public static final ExecutorService pool= Executors.newFixedThreadPool(4);
    public static void main(String[] args) {
        for (int i = 1; i <= 4; i++) {
            int index = i;
            pool.submit(() -> MPSC_ARRAY_QUEUE.offer("疯狂创客圈 java 卷王 " +
index) ); // 入队操作
        }
        try {
            Thread.sleep(1000L);
            MPSC_ARRAY_QUEUE.add("疯狂创客圈 java 卷王 5"); // 入队操作，满则跑出异常
        } catch (Exception e) {
            e.printStackTrace();
        }
        System.out.println("队列大小: " + MPSC_ARRAY_QUEUE.size() + ", 队列容量: "
+ MPSC_ARRAY_QUEUE.capacity());
        System.out.println("出队: " + MPSC_ARRAY_QUEUE.remove()); // 出队操作，队列为空则抛出异常
        System.out.println("出队: " + MPSC_ARRAY_QUEUE.poll()); // 出队操作，队列为空则返回 NULL
    }
}
```

程序输出结果如下：



MpscUnboundedArrayQueue的宏观架构

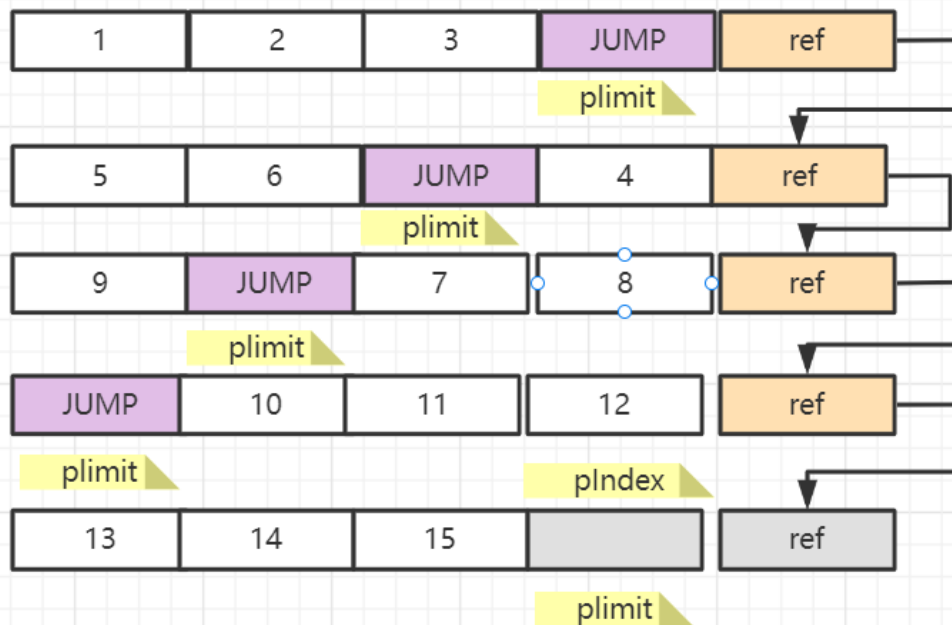
Caffeine和Netty，所使用的并不是原始的MpscArrayQueue，而是其高性能的子类MpscGrowableArrayQueue，来看看其宏观架构。

宏观上，MpscGrowableArrayQueue 基于数组+链表的复合结构。

基于数组+链表的复合结构的优势：

- 1 不会像链表那样分配过多的Node，吞吐量比传统的链表高。
- 2 扩容的时候，也不存在数组复制，扩容的速度，也比传统的数组快

MpscChunkedArrayQueue 基于数组+链表的复合结构



基于数组+链表的复合结构的优势:

- 1 不会像链表那样分配过多的Node, 吞吐量比传统的链表高。
- 2 扩容的时候, 也不存在数组复制, 扩容的速度, 也比传统的数组快。

CSDN @40岁资深老架构师尼恩

MpscUnboundedArrayQueue的重要属性

MpscUnboundedArrayQueue基本的数据结构由「数组+链表」组成, 它有两个指针:

- producerBuffer 指向生产者生产的数组
- consumerBuffer, 指向消费者消费的数组

它还有两个索引指针:

- producerIndex 指向生产者生产的索引
- consumerIndex, 指向消费者消费的索引

这两个索引会以2为步长不断递增。

另外对于生产者, 它还有一个producerLimit指针, 它代表生产者生产消息的上限,

达到该producerLimit上限, Queue就要扩容了,

扩容的方式是创建一个长度一样的新数组, 然后旧数组的最后一个元素指向新数组, 形成单向链表。

这些属性, 为了高性能访问, 都进行了填充, 分布在不同的基类中

- BaseMpscLinkedArrayQueueColdProducerFields 生产者字段

```

abstract class BaseMpscLinkedListArrayQueueColdProducerFields<E> extends
BaseMpscLinkedListArrayQueuePad3<E> {
    private final static long P_LIMIT_OFFSET =
fieldOffset(BaseMpscLinkedListArrayQueueColdProducerFields.class, "producerLimit");

    private volatile long producerLimit; //它代表生产者生产消息的上限
    protected long producerMask; // 计算生产者 数组下标的掩码
    protected E[] producerBuffer; // 计算生产者 数据的 数组

    .....

```

- BaseMpscLinkedListArrayQueueConsumerFields 消费者字段

```

abstract class BaseMpscLinkedListArrayQueueConsumerFields<E> extends
BaseMpscLinkedListArrayQueuePad2<E> {
    private final static long C_INDEX_OFFSET =
fieldOffset(BaseMpscLinkedListArrayQueueConsumerFields.class, "consumerIndex");

    private volatile long consumerIndex; // 消费者索引
    protected long consumerMask; // 计算消费 数组下标的掩码
    protected E[] consumerBuffer; // 计算消费 数据的 数组

    .....

```

看到 mask 变量，是用于计算数组下标的掩码。

相当于取模的操作，只是这里使用位运算取模，所以，队列中数组的容量大小肯定是 2 的次幂。

因为 Mpsc 是多生产者单消费者队列，所以 producerIndex、producerLimit 都是用 volatile 进行修饰的，其中一个生产者线程的修改需要对其他生产者线程可见。

还有一些其他的属性

```

abstract class BaseMpscLinkedListArrayQueue<E> extends
BaseMpscLinkedListArrayQueueColdProducerFields<E>
    implements MessagePassingQueue<E>, QueueProgressIndicators {

    // 数组被生产者填满后，会填充一个JUMP，代表队列扩容了，消费者遇到JUMP会消费下一个数组。

    // No post padding here, subclasses must add
    private static final Object JUMP = new Object();

    // 消费者消费完一个完整的数组后，会将最后一个元素设为BUFFER_CONSUMED。
    private static final Object BUFFER_CONSUMED = new Object();

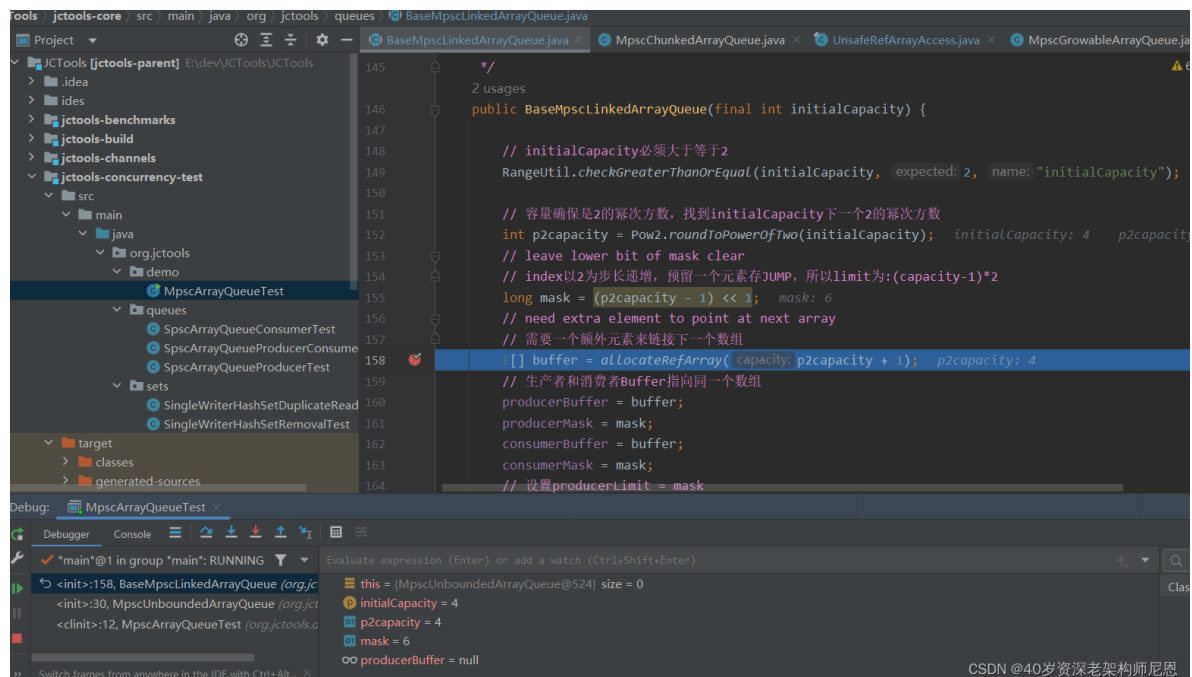
    .....

```

MpscUnboundedArrayQueue构造函数，

需要给定一个chunkSize, 指定块大小,

```
public static final MpscUnboundedArrayQueue<String> MPSC_ARRAY_QUEUE = new
MpscUnboundedArrayQueue<>(4);
```



MpscQueue由一系列数组构成, chunkSize就是数组的大小, 它必须是一个2的幂次方数。

```
public class MpscUnboundedArrayQueue<E> extends BaseMpscLinkedListArrayQueue<E>
{
    long p0, p1, p2, p3, p4, p5, p6, p7;
    long p10, p11, p12, p13, p14, p15, p16, p17;

    public MpscUnboundedArrayQueue(int chunkSize)
    {
        super(chunkSize);
    }
    ....
}
```

父类的构造函数

```
public BaseMpscLinkedListArrayQueue(final int initialCapacity) {

    // initialCapacity必须大于等于2
    RangeUtil.checkGreaterThanOrEqual(initialCapacity, 2,
    "initialCapacity");

    // 容量确保是2的幂次方数, 找到initialCapacity下一个2的幂次方数
    int p2capacity = Pow2.roundToPowerOfTwo(initialCapacity);
    // leave lower bit of mask clear
    // index以2为步长递增, 预留一个元素存JUMP, 所以limit为:(capacity-1)*2
    long mask = (p2capacity - 1) << 1;
    // need extra element to point at next array
    // 需要一个额外元素来链接下一个数组
    E[] buffer = allocateRefArray(p2capacity + 1);
```

```

// 生产者和消费者buffer指向同一个数组
producerBuffer = buffer;
producerMask = mask;
consumerBuffer = buffer;
consumerMask = mask;
// 设置producerLimit = mask
soProducerLimit(mask); // we know it's all empty to start with
}

```

在父类的构造函数中，计算了mask，初始化了一个数组，

并将producerBuffer和consumerBuffer都指向了同一个数组，然后根据mask设置producerLimit。

假设initialCapacity为4，数组的长度就是5，

因为最后一个元素会用来存放扩容数组的地址，形成链表。

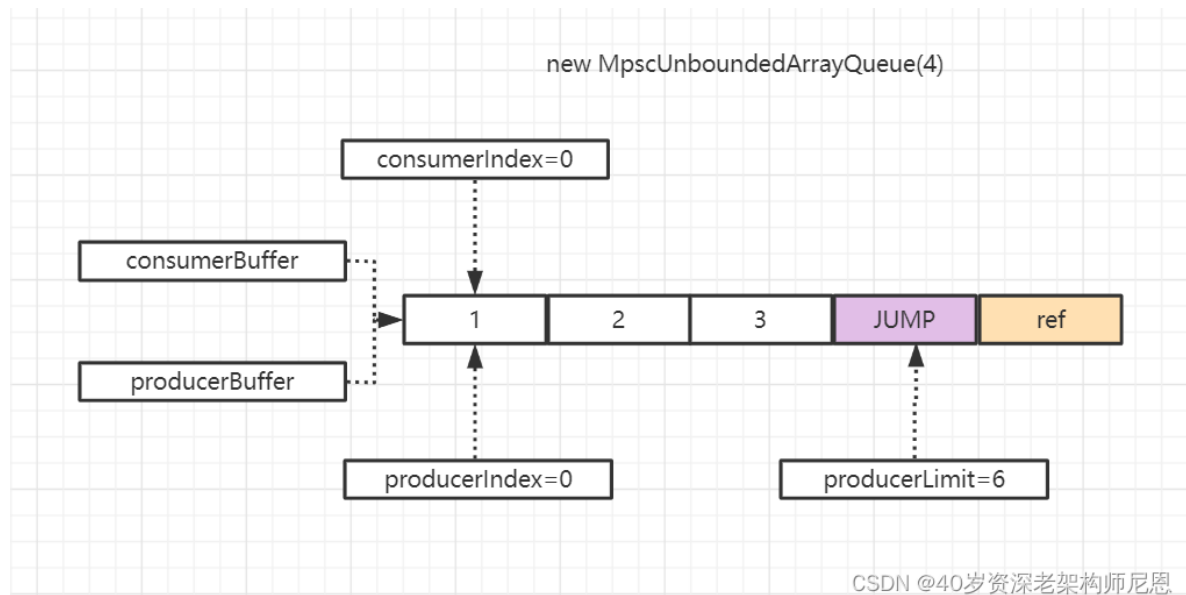
每个数组还会预留一个槽位存放 JUMP 元素，代表队列扩容了，消费者遇到 JUMP 元素就会通过最后一个元素找到扩容后的数组继续消费，因此一个数组最多保留3个元素。

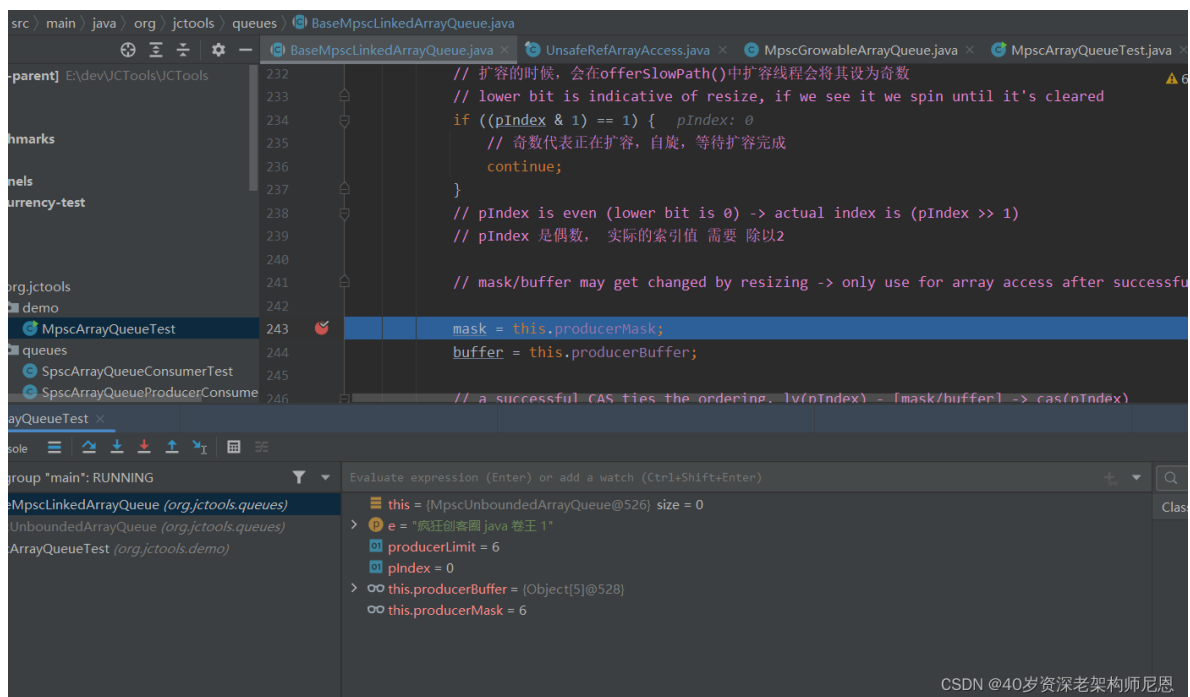
入队、扩容源码分析

图解：入队的核心流程

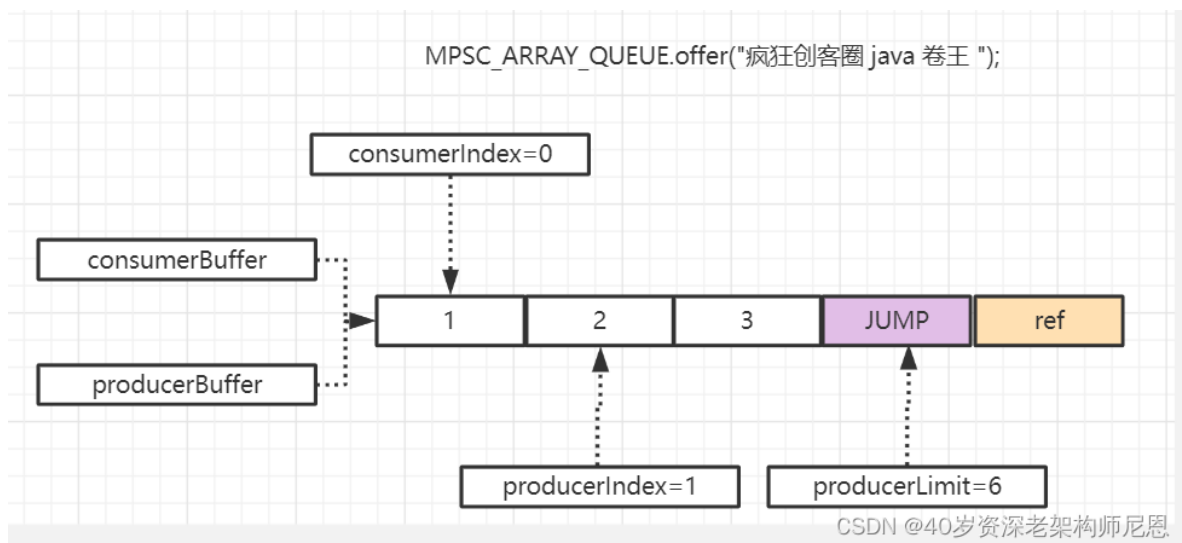
这个结构比较复杂，大家慢慢看吧

新队列的属性值

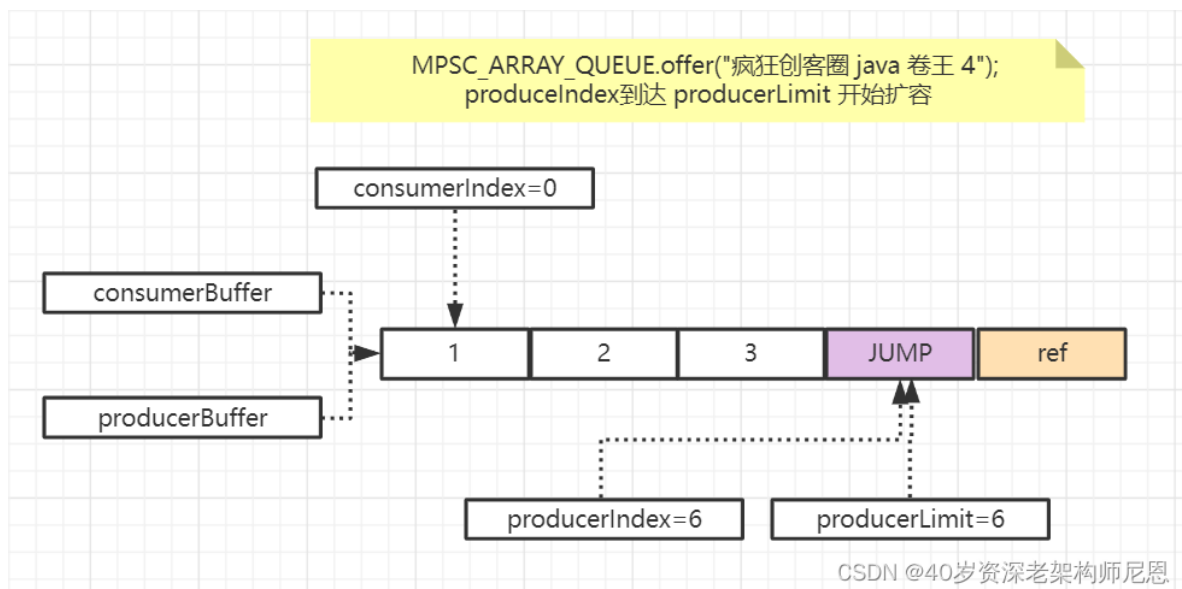




通过offer插入一个元素



第一次produceIndex到达 producerLimit 开始扩容



```
267 resize(mask, buffer, pIndex, e, s, null); e: "疯狂创客圈 java 卷王 4"
268 return true;
269 }
270 }
271 // 阈值 producerLimit 没有 小于等于生产者指针位置 pIndex
272 // 阈值 producerLimit 大于 生产者指针位置 pIndex
273 // 直接通过CAS操作对pIndex做加2处理, 并且退出循环
274 if (casProducerIndex(pIndex, newValue: pIndex + 2)) {
275     break;
276 }
```

Debugger Console:

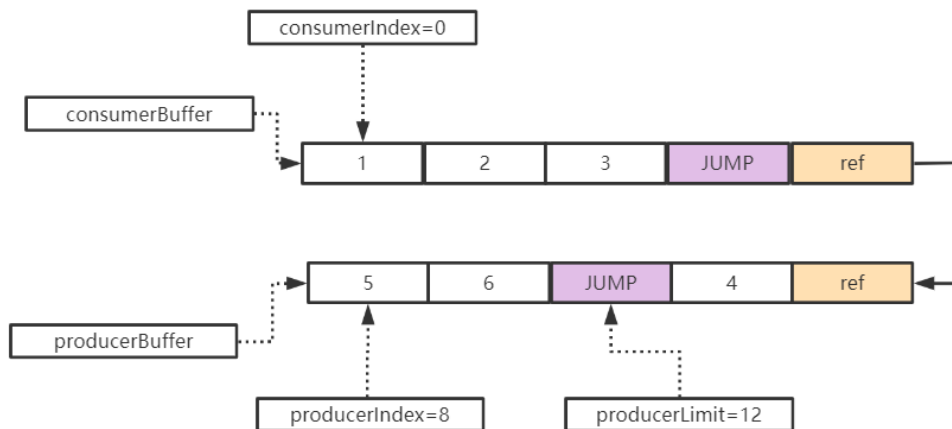
```
offer:267, BaseMpscLinkedArrayQueue (org.jctools)
offer:23, MpscUnboundedArrayQueue (org.jctools)
main:21, MpscArrayQueueTest (org.jctools.demos)

> this = [MpscUnboundedArrayQueue@526] size = 3
> e = "疯狂创客圈 java 卷王 4"
> producerLimit = 6
> pIndex = 6
> mask = 6
> buffer = [Object[5]@528]
  Not showing null elements
  > 0 = "疯狂创客圈 java 卷王 1"
  > 1 = "疯狂创客圈 java 卷王 2"
  > 2 = "疯狂创客圈 java 卷王 3"
> result = 3
```

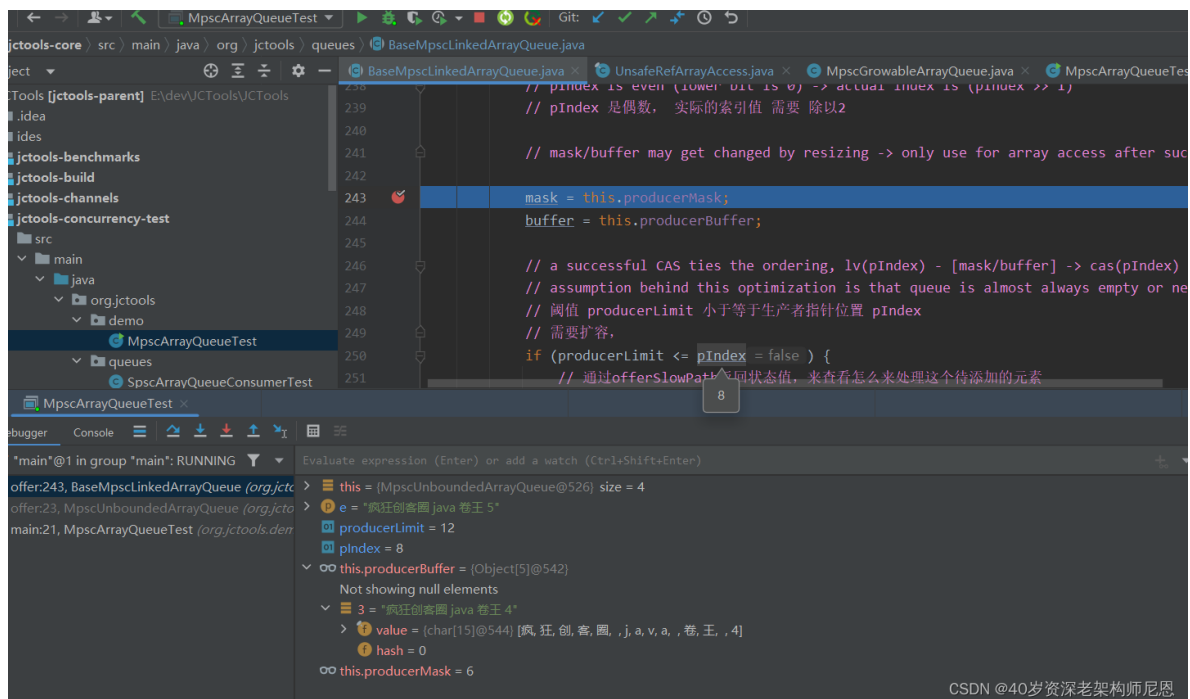
CSDN @40岁资深老架构师尼恩

扩容之后, 插入一个新的元素, 之后的属性值

MPSC_ARRAY_QUEUE.offer("疯狂创客圈 java 卷王 4");
produceIndex到达 producerLimit 开始扩容

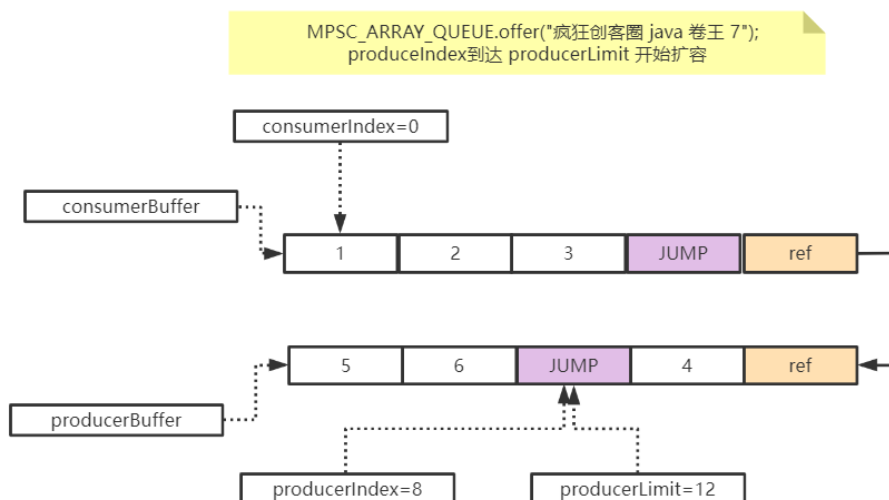
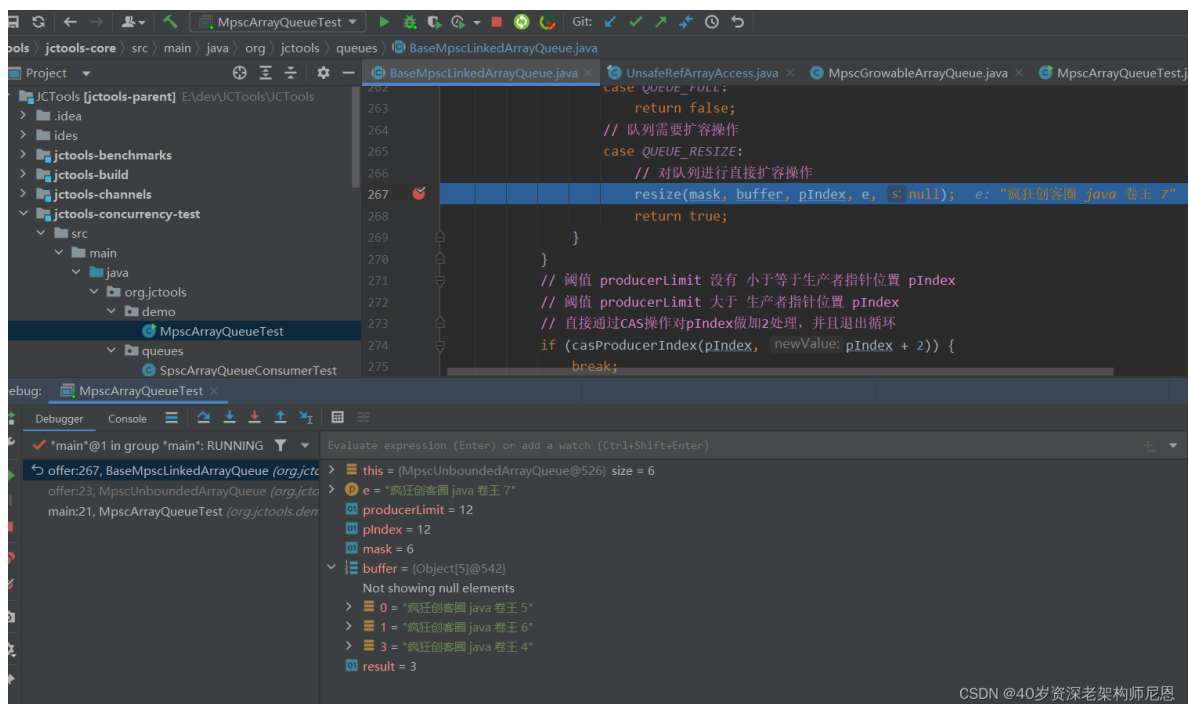


CSDN @40岁资深老架构师尼恩

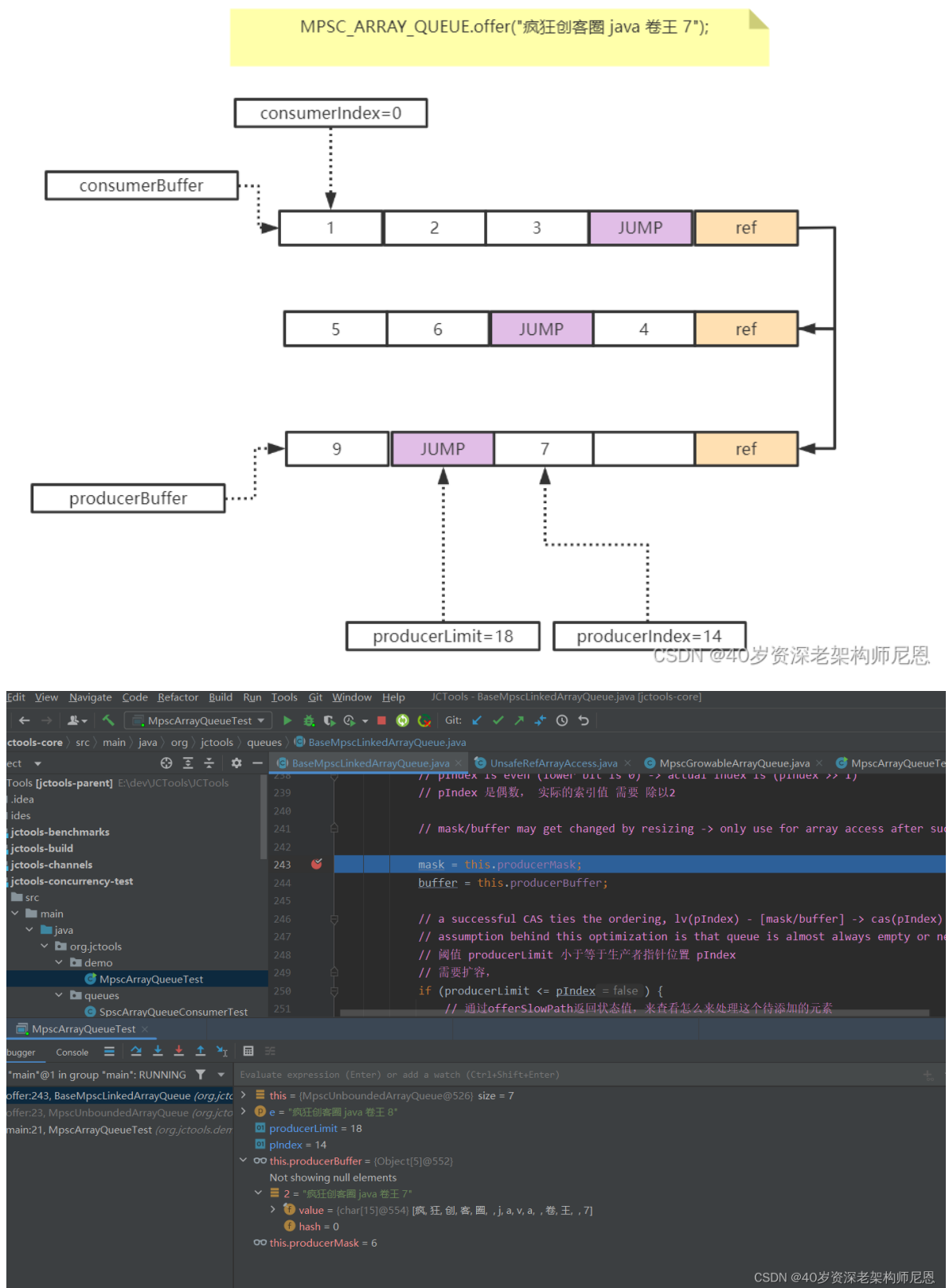


CSDN @40岁资深老架构师尼恩

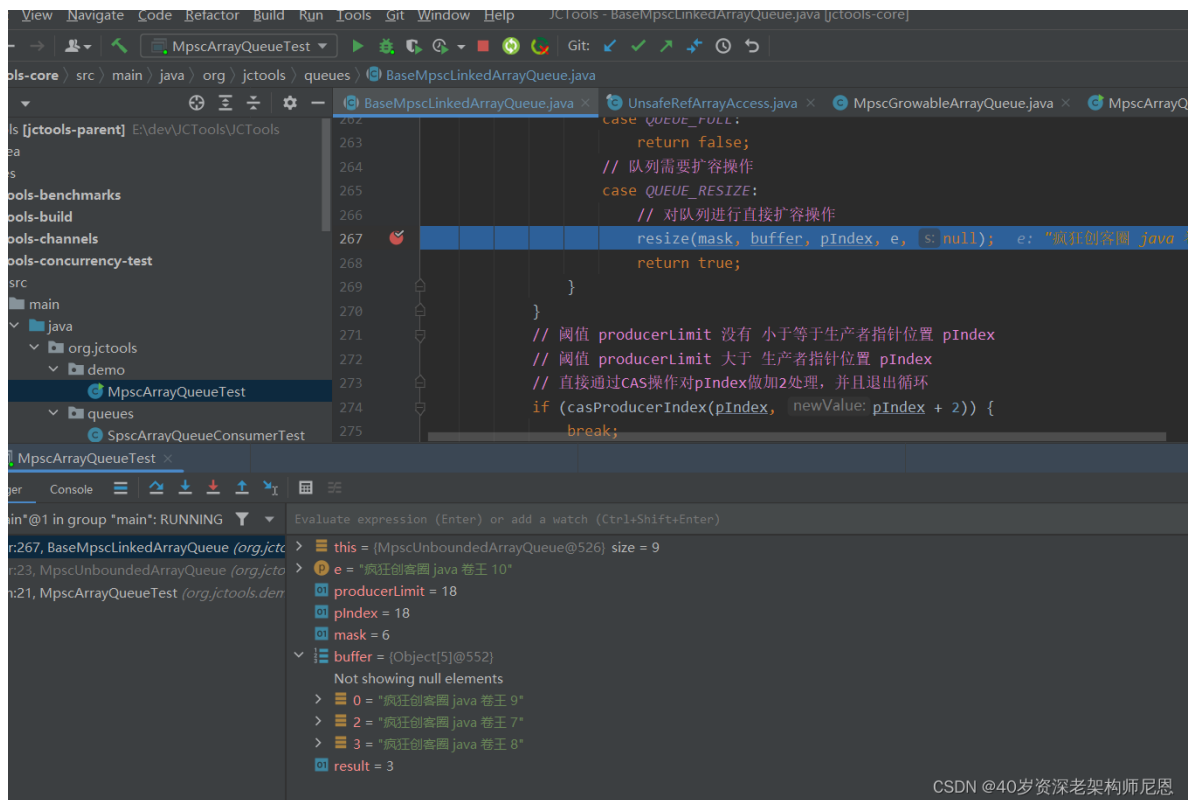
第二次produceIndex到达 producerLimit



扩容之后，插入一个新的元素，之后的属性值

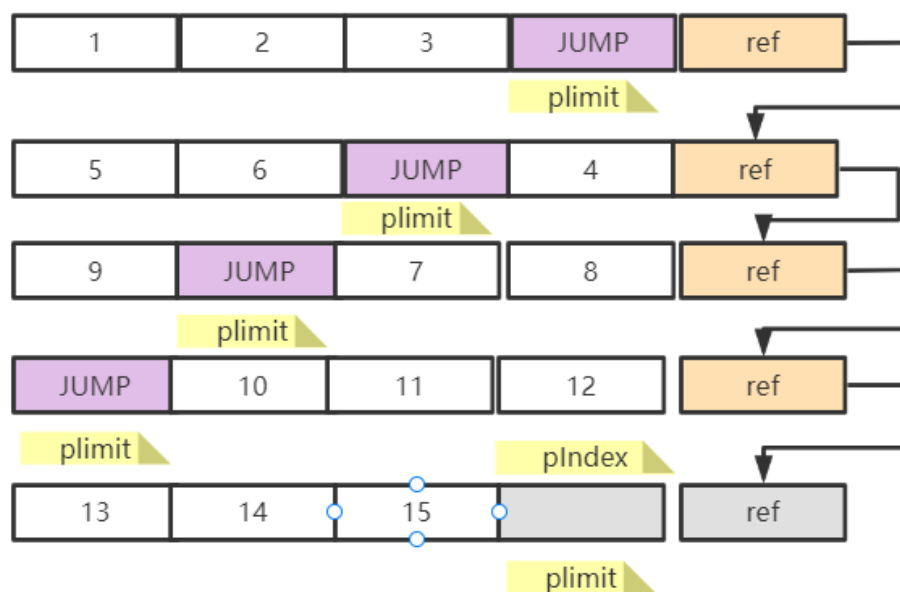


再插入两个元素



CSDN @40岁资深老架构师尼恩

MpscChunkedArrayQueue 基于数组+链表的复合结构



基于数组+链表的复合结构的优势:

- 1 不会像链表那样分配过多的Node, 吞吐量比传统的链表高。
- 2 扩容的时候, 也不存在数组复制, 扩容的速度, 也比传统的数组快

CSDN @40岁资深老架构师尼恩

入队 offer 的源码分析

`offer(e)` 会将元素`e`添加到队列中, 即生产数据。

在MpscQueue中, 线程通过CAS的方式以步长为2递增`producerIndex`, CAS会保证只有一个线程操作成功, CAS成功就代表线程抢到了数组中的槽位, 它可以将元素`e`添加到数组的指定槽位。

CAS失败代表并发失败了，会自旋重试。

如果producerIndex达到producerLimit，代表生产达到上限，队列可能需要扩容了。

offerSlowPath() 方法会判断队列是否需要扩容，如果需要扩容，也只会交给一个线程去扩容，这里又是一个CAS操作，线程以1为步长递增producerIndex，只有CAS成功的线程才会去执行扩容逻辑。

因此，在 offer(e) 的逻辑中，还会判断producerIndex是否是奇数，如果为奇数就代表队列正在扩容。

因为MpscQueue的扩容非常快，它不需要迁移元素，只需要创建一个新数组，再和旧数组建立连接就可以了，

所以没有必要让其他线程挂起，线程发现队列在扩容时，会进行自旋重试，直到扩容完成。

```
/**
 * , 生产数据
 * 向队列中添加一个元素e
 *
 * @param e not {@code null}, will throw NPE if it is
 * @return
 */
@Override
public boolean offer(final E e) {
    if (null == e) {
        throw new NullPointerException();
    }

    long mask;
    E[] buffer; //生产者指向的数组
    long pIndex; //生产索引

    while (true) {
        long producerLimit = lvProducerLimit(); // 获取生产者索引最大限制

        pIndex = lvProducerIndex(); // 获取生产者索引

        // 生产索引以2为步长递增，
        // 第0位标识为resize，所以非扩容场景，不会是奇数，
        // 扩容的时候，会在offerSlowPath()中扩容线程会将其设为奇数
        // lower bit is indicative of resize, if we see it we spin until
it's cleared
        if ((pIndex & 1) == 1) {
            // 奇数代表正在扩容，自旋，等待扩容完成
            continue;
        }
        // pIndex is even (lower bit is 0) -> actual index is (pIndex >> 1)
        // pIndex 是偶数， 实际的索引值 需要 除以2

        // mask/buffer may get changed by resizing -> only use for array
access after successful CAS.

        mask = this.producerMask;
        buffer = this.producerBuffer;

        // a successful CAS ties the ordering, lv(pIndex) - [mask/buffer] ->
cas(pIndex)
```

```

        // assumption behind this optimization is that queue is almost
always empty or near empty
        // 阈值 producerLimit 小于等于生产者指针位置 pIndex
        // 需要扩容，
        if (producerLimit <= pIndex) {
            // 通过offersSlowPath返回状态值，来查看怎么来处理这个待添加的元素
            int result = offersSlowPath(mask, pIndex, producerLimit);
            switch (result) {
                // producerLimit虽然达到了limit，
                // 但是当前数组已经被消费了部分数据，暂时不会扩容，会使用已被消费的槽
                // 位。

                case CONTINUE_TO_P_INDEX_CAS:
                    break;
                // 可能由于并发原因导致CAS失败，那么则再次重新尝试添加元素
                case RETRY:
                    continue;
                    // 队列已满，直接返回false操作
                case QUEUE_FULL:
                    return false;
                // 队列需要扩容操作
                case QUEUE_RESIZE:
                    // 对队列进行直接扩容操作
                    resize(mask, buffer, pIndex, e, null);
                    return true;
            }
        }
        // 阈值 producerLimit 大于 生产者指针位置 pIndex
        // 直接通过CAS操作对pIndex做加2处理
        if (casProducerIndex(pIndex, pIndex + 2)) {
            break;
        }
    }
    // INDEX visible before ELEMENT
    final long offset = modifiedCalcCircularRefElementOffset(pIndex, mask);
    // 将buffer数组的指定位置替换为e，
    // 不是根据下标来设置的，是根据槽位的地址偏移量offset，UNSAFE操作。
    soRefElement(buffer, offset, e); // release element e
    return true;
}

```

`offersSlowPath()` 会告诉线程队列是满了，还是需要扩容，还是需要自旋重试。

总之，这个一条慢路径，所以叫做 slow path。

虽然producerIndex达到了producerLimit，但不代表队列就非得扩容，

如果消费者已经消费了部分生产者指向的数组元素，就意味这当前数组还是有槽位可以继续用的，暂时不用扩容。

```

/**
 * @param mask
 * @param pIndex 生产者索引
 * @param producerLimit 生产者limit
 * @return
 */
private int offersSlowPath(long mask, long pIndex, long producerLimit) {
    // 消费者索引

```

```

final long cIndex = lvConsumerIndex();
// 数组缓冲的容量, (长度-1) * 2
long bufferCapacity = getCurrentBufferCapacity(mask);
// 消费索引 + 当前数组的容量 > 生产索引, 代表当前数组已有部分元素被消费了,
// 不会扩容, 会使用已被消费的槽位。
// cIndex + bufferCapacity ==> producerLimit
if (cIndex + bufferCapacity > pIndex) {
    if (!casProducerLimit(producerLimit, cIndex + bufferCapacity)) {
        // CAS失败, 自旋重试
        // retry from top
        return RETRY;
    } else {
        // continue to pIndex CAS
        // 重试 CAS修改 生产索引
        return CONTINUE_TO_P_INDEX_CAS;
    }
}
// full and cannot grow
// 根据生产者和消费者索引判断Queue是否已满, 无界队列永不会满
else if (availableInQueue(pIndex, cIndex) <= 0) {
    // offer should return false;
    return QUEUE_FULL;
}
// grab index for resize -> set lower bit
// CAS的方式将producerIndex加1, 奇数代表正在resize
else if (casProducerIndex(pIndex, pIndex + 1)) {
    // trigger a resize
    return QUEUE_RESIZE;
} else {
    // failed resize attempt, retry from top
    return RETRY;
}
}

```

如果需要扩容, 线程会CAS操作将producerIndex改为奇数, 让其它线程能感知到队列正在扩容, 要生产数据的线程先自旋, 等待扩容完成再继续操作。

offer()过程中, 通过CAS抢槽位, CAS失败的线程自旋重试。

如果遇到队列需要扩容, 则将producerIndex设为奇数, 其他线程自旋等待, 一直等到扩容完成, 扩容后再设为偶数, 通知其它线程继续生产。

入队扩容源码分析

resize() 是扩容的核心方法,

它首先会创建一个相同长度的新数组, 将producerBuffer指向新数组, 然后将元素e放到新数组中,

旧元素的最后一个元素指向新数组, 形成链表。

还会将旧元素的槽位填充 JUMP 元素, 代表队列扩容了。

```

// 扩容:
// 新建一个E[], 将oldBuffer和newBuffer建立连接。
// 将producerBuffer指向新数组, 然后将元素e放到新数组中,
// 旧元素的最后一个元素指向新数组, 形成链表。
// 还会将旧元素的槽位填充JUMP元素, 代表队列扩容了。

```

```

private void resize(long oldMask, E[] oldBuffer, long pIndex, E e,
Supplier<E> s) {
    assert (e != null && s == null) || (e == null || s != null);

    // 下一个Buffer的长度, MpscQueue会构建一个相同长度的Buffer
    int newBufferLength = getNextBufferSize(oldBuffer);
    final E[] newBuffer;
    try {
        // 创建一个新的E[]

        newBuffer = allocateRefArray(newBufferLength);
    } catch (OutOfMemoryError oom) {
        assert lvProducerIndex() == pIndex + 1;
        soProducerIndex(pIndex);
        throw oom;
    }

    // 生产者Buffer指向新的E[]
    producerBuffer = newBuffer;
    // 计算新的Mask, Buffer长度不变的情况下, Mask也不变
    final int newMask = (newBufferLength - 2) << 1;
    producerMask = newMask;

    // 根据该偏移量设置oldBuffer的JUMP元素, 会递增然后重置, 不断循环
    final long offsetInOld = modifiedCalcCircularRefElementOffset(pIndex,
oldMask);
    // Mask不变的情况下, oldBuffer的JUMP对应的位置, 就是newBuffer中要消费的位置.
    final long offsetInNew = modifiedCalcCircularRefElementOffset(pIndex,
newMask);

    // 元素e放到新数组中
    soRefElement(newBuffer, offsetInNew, e == null ? s.get() : e); // element
in new array
    // 旧数组和新数组建立连接, 旧数组的最后一个元素保存新数组的地址。
    soRefElement(oldBuffer, nextArrayOffset(oldMask), newBuffer); // buffer
linked

    // ASSERT code
    final long cIndex = lvConsumerIndex();
    final long availableInQueue = availableInQueue(pIndex, cIndex);
    RangeUtil.checkPositive(availableInQueue, "availableInQueue");

    // Invalidate racing CASS
    // We never set the limit beyond the bounds of a buffer
    soProducerLimit(pIndex + Math.min(newMask, availableInQueue));

    // make resize visible to the other producers
    soProducerIndex(pIndex + 2);

    // INDEX visible before ELEMENT, consistent with consumer expectation

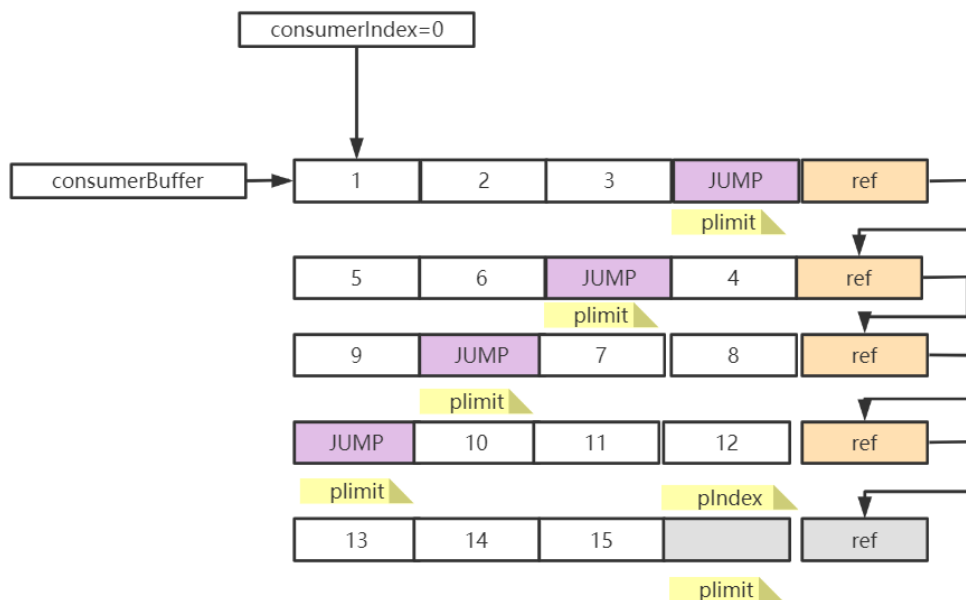
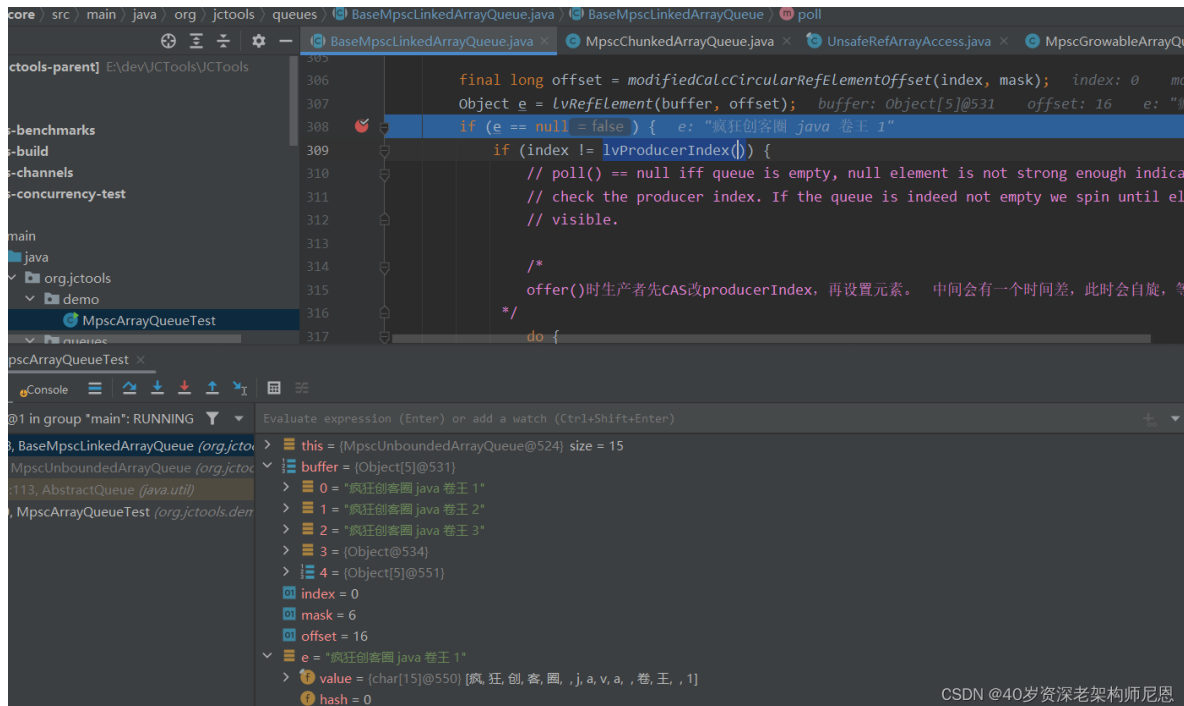
    // make resize visible to consumer
    soRefElement(oldBuffer, offsetInOld, JUMP);
}

```


出队 poll的核心流程

图解：poll的核心流程

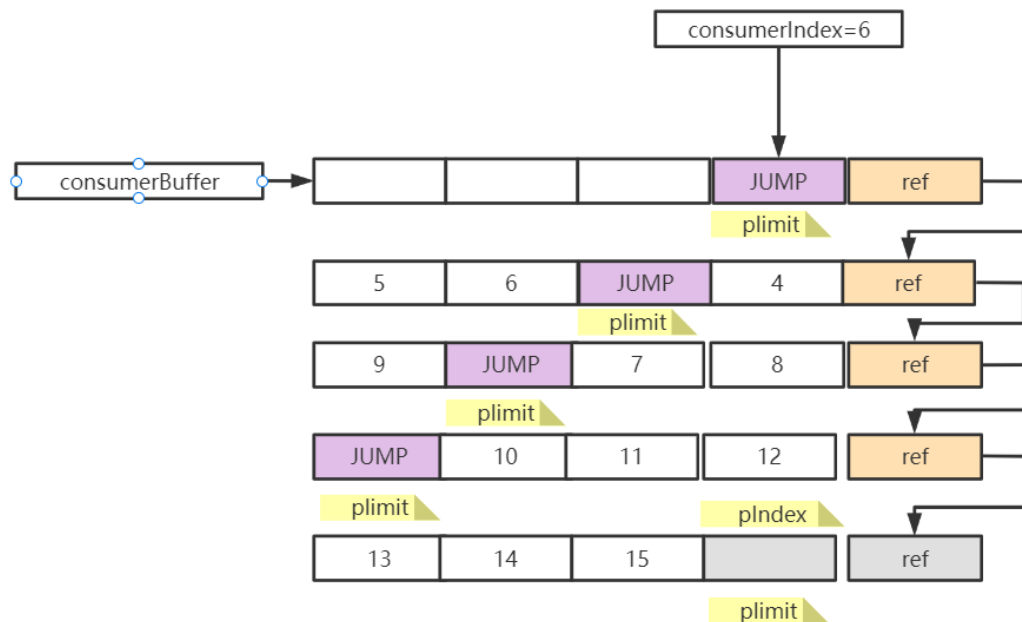
这个结构比较复杂，大家慢慢看吧



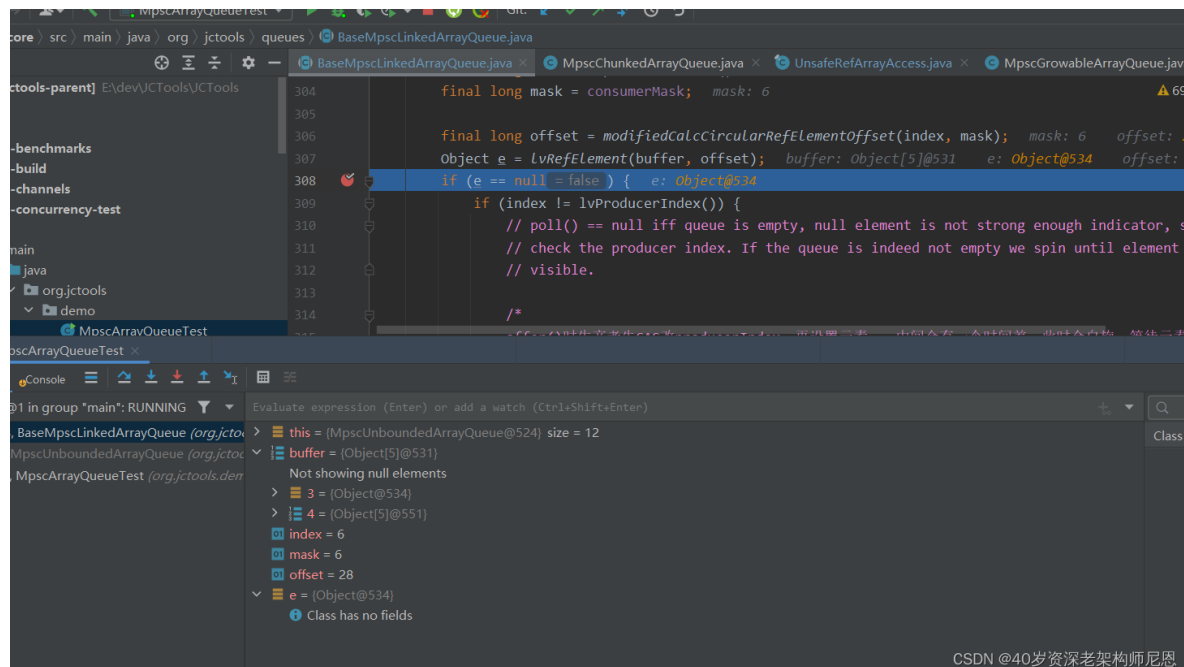
queue.poll() 消费后将槽位设null

一个Buffer消费完毕，消费遇到JUMP节点

一个Buffer消费完毕，消费遇到JUMP节点，

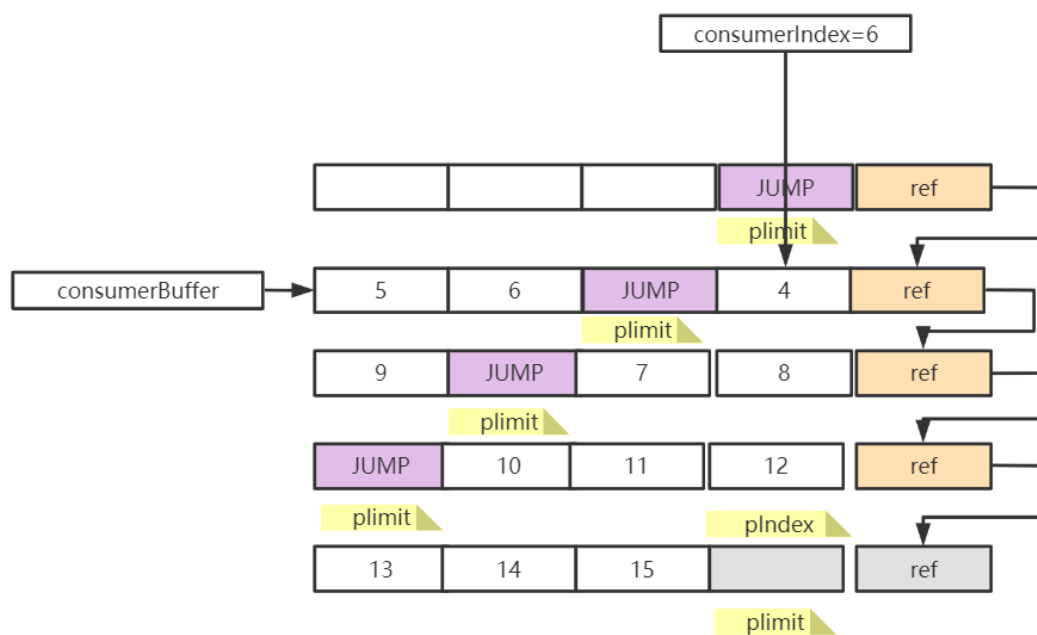
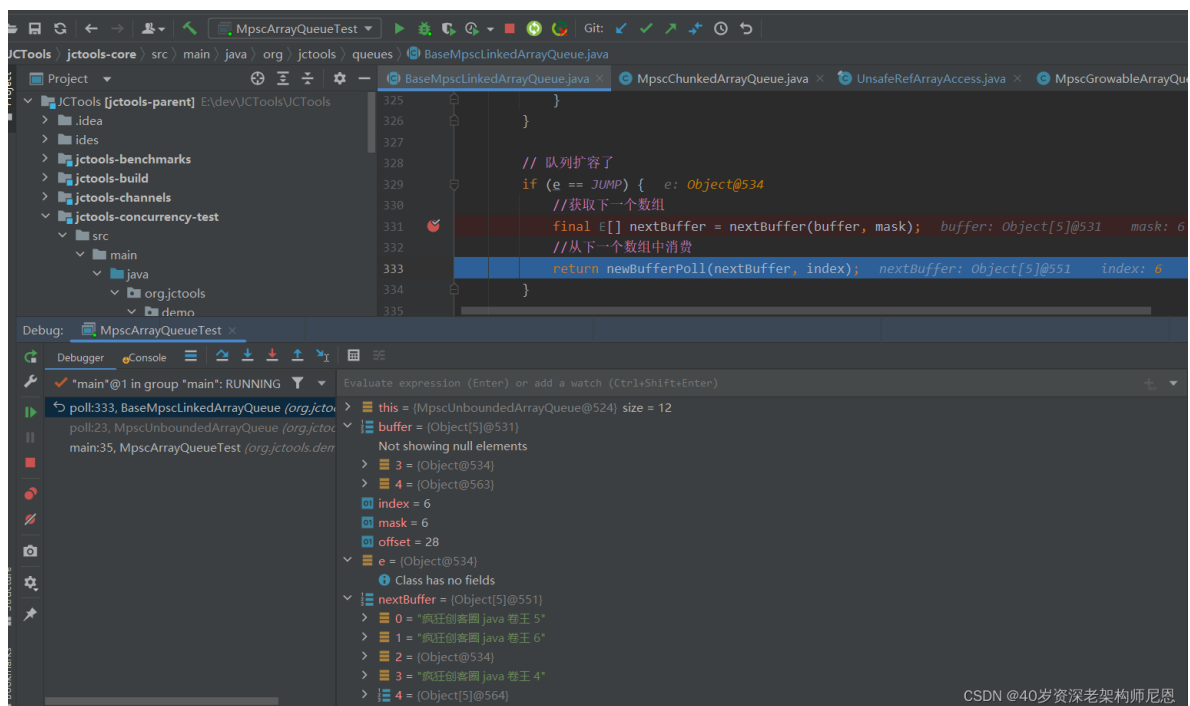


CSDN @40岁资深老架构师尼恩



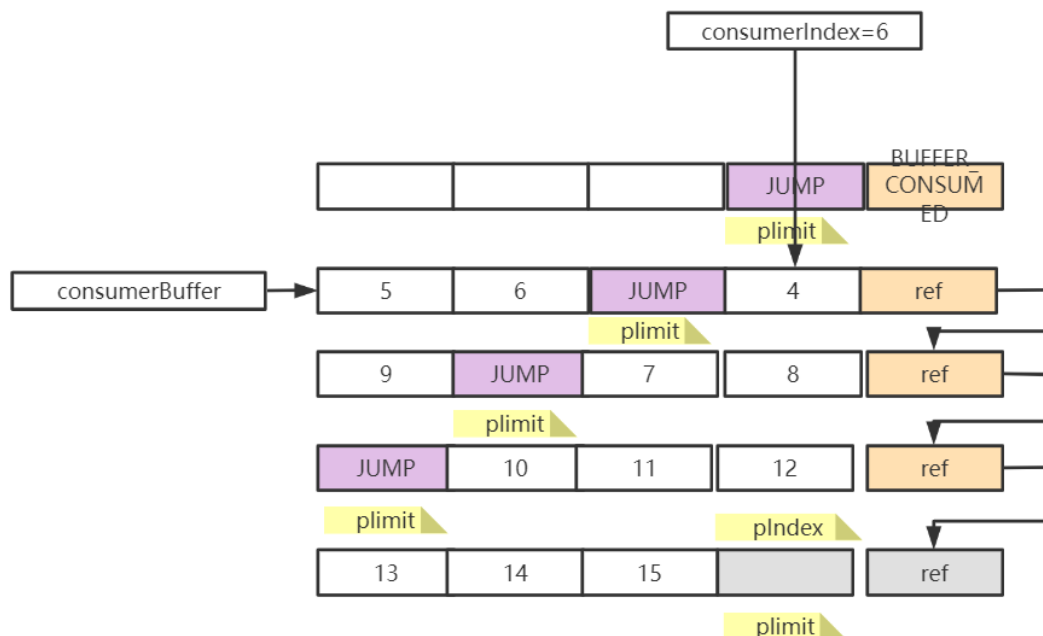
CSDN @40岁资深老架构师尼恩

通过nextbuffer找下一个数组，消费相同索引位的元素



CSDN @40岁资深老架构师尼恩

整个数组消费完了，将最后一位设为BUFFER CONSUMED，从链表中剔除。



CSDN @40岁资深老架构师尼恩

出队 poll的源码分析

poll() 方法核心思路是获取消费者索引 consumerIndex，然后根据 consumerIndex 计算得出数组对应的偏移量，然后将数组对应位置的元素取出并返回，最后将 consumerIndex 移动到环形数组下一个位置。

如果元素为null，并不代表队列是空的，还要比较consumerIndex和producerIndex，如果两者索引不同，那么producerIndex肯定是大于consumerIndex的，说明生产者已经在生产了，移动了producerIndex，只是还没来得及将元素填充到数组而已。

因为生产者是先CAS递增producerIndex，再将元素填充到数组的，两步之间存在一个非常短的时间差，如果消费者恰好在这个时间差内去消费数据，那么就自旋等待一下，等待生产者填充元素到数组。

如果元素为JUMP，说明队列扩容了，消费者需要根据数组的最后一个元素找到扩容后的新数组，消费新数组的元素。

```
// poll()没有做并发控制，MpscQueue是多生产单消费者的Queue，只有一个消费者，这个也是
netty 换成 mpscqueue的主要原因
@Override
public E poll() {
    final E[] buffer = consumerBuffer;
    final long index = lpConsumerIndex();
    final long mask = consumerMask;

    final long offset = modifiedCalcCircularRefElementOffset(index, mask);
    Object e = lvRefElement(buffer, offset);
    if (e == null) {
        if (index != lpProducerIndex()) {
            // poll() == null iff queue is empty, null element is not strong
            enough indicator, so we must
            // check the producer index. If the queue is indeed not empty we
            spin until element is
            // visible.

            /*
```

`offer()`时生产者先CAS改`producerIndex`，再设置元素。中间会有一个时间差，此时会自旋，等待元素设置完成。

```
        */
        do {
            e = lvRefElement(buffer, offset);
        }
        while (e == null);
    } else {

        //元素已经消费完
        return null;
    }
}

// 队列扩容了
if (e == JUMP) {
    //获取下一个数组
    final E[] nextBuffer = nextBuffer(buffer, mask);
    //从下一个数组中消费
    return newBufferPoll(nextBuffer, index);
}

// 取出元素后，将原来的槽位设为null
soRefElement(buffer, offset, null); // release element null
// 递增consumerIndex
soConsumerIndex(index + 2); // release cIndex
return (E) e;
}
```

如果队列扩容了，`nextBuffer()`会找到扩容后的新数组，同时它还会将旧数组的最后一个元素设为`BUFFER_CONSUMED`，代表当前数组已经被消费完了，也就从链表中剔除了。

```
@SuppressWarnings("unchecked")
private E[] nextBuffer(final E[] buffer, final long mask) {

    /**
     * 通过当前数组的最后一个元素，获取下一个待消费的数组，
     * 将当前数组最后一个元素设为BUFFER_CONSUMED，代表当前数组已经消费完毕。
     */
    final long offset = nextArrayOffset(mask);
    final E[] nextBuffer = (E[]) lvRefElement(buffer, offset);
    consumerBuffer = nextBuffer;
    consumerMask = (length(nextBuffer) - 2) << 1;
    soRefElement(buffer, offset, BUFFER_CONSUMED);
    return nextBuffer;
}
```

队列中的有序写入、有序读取

有序写入数组元素

offer写入的时候，jctool根据 pIndex 进行位运算计算得到数组对应的下标，然后通过 soRefElement方法将数据写入到数组中，源码如下所示。

```
    }  
    // INDEX visible before ELEMENT  
    final long offset = modifiedCalcCircularRefElementOffset(pIndex, mask);  
    // 将buffer数组的指定位置替换为e，  
    // 不是根据下标来设置的，是根据槽位的地址偏移量offset，UNSAFE操作。  
    soRefElement(buffer, offset, e); // An ordered store of an element to a  
    given offset  
    return true;
```

```
/**  
 * An ordered store of an element to a given offset  
 *  
 * @param buffer this.buffer  
 * @param offset computed via {@link  
UnsafeRefArrayAccess#calcCircularRefElementOffset}  
 * @param e      an orderly kitty  
 */  
public static <E> void soRefElement(E[] buffer, long offset, E e)  
{  
    UNSAFE.putOrderedObject(buffer, offset, e);  
}
```

putOrderedObject() 和 putObject() 都可以用于更新对象的值，但是 putOrderedObject() 并不会立刻将数据更新到内存中，同时，也不会去保障不同CPU内核的 Cache Line 数据强一致。

这部分原理比较复杂，具体请参见《Java高并发核心编程卷2》。

putOrderedObject() 使用的是 LazySet 延迟更新机制，所以性能方面 putOrderedObject() 要比 putObject() 高很多。

Java 中有四种类型的内存屏障，分别为 LoadLoad、StoreStore、LoadStore 和 StoreLoad。

putOrderedObject() 使用了 StoreStore Barrier，对于 Store1，StoreStore，Store2 这样的操作序列，在 Store2 进行写入之前，会保证 Store1 的写操作对其他处理器可见。

有序写入的优势，就是性能高，写操作结果有纳秒级的延迟。

但是，有序写入是有代价的，不会立刻被其他线程以及自身线程可见。

因为在 Mpsc Queue 的使用场景中，多个生产者只负责写入数据，并没有写入之后立刻读取的需求，所以使用 LazySet 机制是没有问题的，只要 StoreStore Barrier 保证多线程写入的顺序即可。

虽然有序写入数组元素，但是，对生产者的pIndex的写入，用的是cas写入，这个是有强一致性的。通过StoreLoad Barrier保证有序性和可见性。

```
44 // lv = lazy value
45 @Override
46 public final long lvProducerIndex() {
47     return producerIndex;
48 }
49
50 // so = set ordered / lazy set
51 // 2 usages
52 final void soProducerIndex(long newValue) {
53     UNSAFE.putOrderedLong( o: this, P_INDEX_OFFSET, newValue);
54 }
55 // 3 usages
56 final boolean casProducerIndex(long expect, long newValue) {
57     return UNSAFE.compareAndSwapLong( o: this, P_INDEX_OFFSET, expect, newValue);
58 }
59
60 // 1 usage 8 inheritors
61 abstract class BaseMpscLinkedArrayQueuePad2<E> extends BaseMpscLinkedArrayQueueProducerFields<E> {
62     long p01, p02, p03, p04, p05, p06, p07;
63     long p10, p11, p12, p13, p14, p15, p16, p17;
```

因为生产者有多个线程，所以 MpscArrayQueue 采用了 UNSAFE.getLongVolatile() 方法保证获取 pIndex 索引，保证其可见性、准确性。

```
224
225 long mask;
226 E[] buffer; //生产者指向的数组
227 long pIndex; //生产索引
228
229 while (true) {
230     long producerLimit = lvProducerLimit(); // 获取生产者索引最大限制
231
232     pIndex = lvProducerIndex(); // 获取生产者索引
233
234     // 生产索引以2为步长递增，
235     // 第0位标识为resize，所以非扩容场景，不会是奇数，
236     // 扩容的时候，会在offerSlowPath()中扩容线程会将其设为奇数
237     // lower bit is indicative of resize, if we see it we spin until it's cleared
238     if ((pIndex & 1) == 1) {
239         // 奇数代表正在扩容，自旋，等待扩容完成
240         continue;
241     }
242     // pIndex is even (lower bit is 0) -> actual index is (pIndex >> 1)
243     // pIndex 是偶数，实际的索引值 需要 除以2
244
245     // mask/buffer may get changed by resizing -> only use for array access after success
```

```
caffeine / src / main / java / com / github / benmanes / caffeine / cache / MpscGrowArrayQueue.java / BaseMpscLinkedArrayQueue / lvProducerIndex
BlockingQueue.java x BlockingQueue x ArrayBlockingQueue.java x MpscGrowArrayQueueTest.java x MpscGrowArrayQueue
560 /**
561  * @return current buffer capacity for elements (excluding next pointer and jump entry) * 2
562  */
563 1 usage 2 implementations coder001
564 protected abstract long getCurrentBufferCapacity(long mask);
565 6 usages coder001
566 static long lvProducerIndex(BaseMpscLinkedArrayQueue<?> self) {
567     return (long) P_INDEX.getVolatile(self);
568 }
569 6 usages coder001
570 static long lvConsumerIndex(BaseMpscLinkedArrayQueue<?> self) {
571     return (long) C_INDEX.getVolatile(self);
572 }
573 1 usage coder001
574 static void soProducerIndex(BaseMpscLinkedArrayQueue<?> self, long v) { P_INDEX.setRelease(self, v); }
575 2 usages coder001
576 static boolean casProducerIndex(BaseMpscLinkedArrayQueue<?> self, long expect, long newValue) {
577     return P_INDEX.compareAndSet(self, expect, newValue);
578 }
579 3 usages coder001
CSDN @40岁资深老架构师尼恩
```

getLongVolatile() 使用了 StoreLoad Barrier，对于 Store1，StoreLoad，Load2 的操作序列，在 Load2 以及后续的读取操作之前，都会保证 Store1 的写入操作对其他处理器可见。

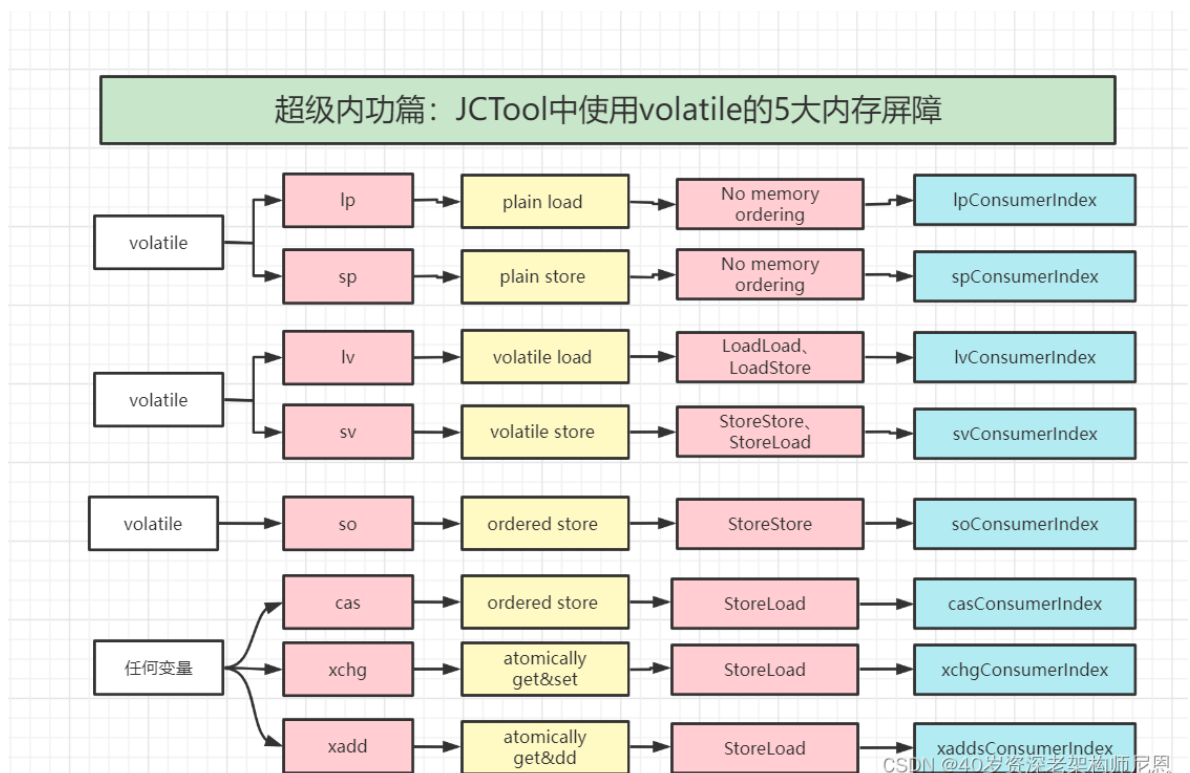
StoreLoad 是四种内存屏障开销最大的，

JCTool中的宽松读写命名规约

排他读写：这里把对volatile 变量的可见写入和可见读取，叫做排他读写，需要保证 强一致，性能低

宽松读写：这里把对volatile 变量的有序写入和有序读取，叫做宽松读写，不需要保证 强一致，但是纳秒级操作

JCTool中的宽松读写命名规约，如下：



MpscArrayQueue 的优点总结：

MpscQueue由一系列数组构成，数组的最后一个元素指向下一个数组，形成单向链表。

MpscQueue全程无锁化，非阻塞，相较于JDK提供的同步阻塞队列，性能有很好的提升，这也是Netty后来的版本将任务队列替换为JCTools的重要原因。

在生产过程中，它用到了大量的CAS操作，对于需要做并发控制的地方，确保只有一个线程会执行成功，其他CAS失败的线程会自旋重试，全程都是无锁非阻塞的。

不管是扩容，还是等待元素被填充到数组，这些过程都是会极快完成的，因此短暂的自旋会比线程挂起再唤醒效率更高。数组扩容后会在原槽位填充 JUMP 元素，消费者遇到该元素就知道要寻找新数组继续消费了。

- 通过cacheline padding 解决核心属性的伪共享问题。
- 数组的容量设置为 2 的次幂，可以通过位运算快速定位到数组对应下标。
- 入队操作通过CAS无锁编程实现，并且通过链式数组结构，和数组节点的动态增加，解决扩容时的元素复制的问题，完成数组的快速扩容，减少CAS空自旋。
- 在消费者poll过程中，因为只有一个消费者线程，所以整个 poll() 的过程没有 CAS 操作。
- 通过有序的写入元素，去掉volatile 的StoreLoad 屏障，实现纳米级别的写入。当然，读取的时候，如果间隔太短，需要进行短时间的自旋。

这个是非常高性能的。这也是Netty、Caffeine 使用mpsc 队列的原因

参考文献

<https://blog.csdn.net/bz120413/article/details/122107790>

<https://blog.csdn.net/Javaesandyou/article/details/123918852>

<https://blog.csdn.net/Javaesandyou/article/details/123918852>

<https://blog.csdn.net/FreeLinux/article/details/54897192>

https://blog.csdn.net/weixin_41605937/article/details/121972371

推荐阅读：

- 《[尼恩Java面试宝典](#)》
- 《[Springcloud gateway 底层原理、核心实战 \(史上最全\)](#)》
- 《[Flux、Mono、Reactor 实战 \(史上最全\)](#)》
- 《[sentinel \(史上最全\)](#)》
- 《[Nacos \(史上最全\)](#)》
- 《[分库分表 Sharding-JDBC 底层原理、核心实战 \(史上最全\)](#)》

- 《[TCP协议详解 \(史上最全\)](#)》
- 《[clickhouse 超底层原理 + 高可用实操 \(史上最全\)](#)》

- [《nacos高可用（图解+秒懂+史上最全）》](#)
- [《队列之王：Disruptor 原理、架构、源码 一文穿透》](#)
- [《环形队列、条带环形队列 Striped-RingBuffer（史上最全）》](#)
- [《一文搞定：SpringBoot、SLF4j、Log4j、Logback、Netty之间混乱关系（史上最全）](#)
- [《单例模式（史上最全）](#)
- [《红黑树（图解 + 秒懂 + 史上最全）》](#)
- [《分布式事务（秒懂）》](#)
- [《缓存之王：Caffeine 源码、架构、原理（史上最全，10W字 超级长文）》](#)
- [《缓存之王：Caffeine 的使用（史上最全）》](#)
- [《Java Agent 探针、字节码增强 ByteBuddy（史上最全）》](#)
- [《Docker原理（图解+秒懂+史上最全）》](#)
- [《Redis分布式锁（图解 - 秒懂 - 史上最全）》](#)
- [《Zookeeper 分布式锁 - 图解 - 秒懂》](#)
- [《Zookeeper Curator 事件监听 - 10分钟看懂》](#)
- [《Netty 粘包 拆包 | 史上最全解读》](#)
- [《Netty 100万级高并发服务器配置》](#)
- [《Springcloud 高并发 配置（一文全懂）》](#)

硬核推荐：尼恩Java硬核架构班

又名疯狂创客圈社群 VIP

详情：<https://www.cnblogs.com/crazymakercircle/p/9904544.html>



尼恩java
硬核架构班

已经发布

- 《高能性能RPC的基础实操之：从0到1开始IM撸一个IM》
- 《分布式高能性能RPC的基础实操之：千万级用户分布式IM实操-含简历指导》
- 《亿级用户超高并发秒杀实操-含简历指导》
亮点：助力小伙伴搞定70W年薪，N个涨薪50%，2023春招面试涨薪神器
- 《横扫全网，工业级elasticsearch底层原理与高并发、高可用架构实操》
亮点：40岁老架构师细致解读，处处透着分布式、高性能中间件的原理和精髓
- 《第1部曲：超级底层：葵花宝典（高性能秘籍）__架构师视角解读OS操作系统》
亮点：大制作解读OS操作系统，并揭秘mmap、pagecache、zerocopy等底层的底层原理
2023春招面试涨薪大神器
- 《Rocketmq视频第2部曲：横扫全网工业级 rocketmq 高可用（HA）底层原理和实操》
亮点：起底式、绞杀式解读 rocketmq如何保障消息的可靠性？
- 《Rocketmq视频第3部曲：超级内功篇、横扫全网 rocketmq 源码学习以及3高架构模式解读》
亮点：大制作解读 Rocketmq源码以及3高架构模式，助力大家内力猛增
- 《Rocketmq视频第4部曲：10Wqps消息推送中台架构、设计、编码、测试实操》
亮点：Netty实操、分库分表实操、Rocketmq工业级使用实操
- 《架构师内功篇：横扫全网 netty 高性能、高并发架构 底层原理、源码学习》
- 《架构师实操篇：redis cluster 工业级高可用实操》
- 《架构师实操篇：100W级别QPS日志平台实操》
- 《彻底穿透：skywalking 源码(代表链路跟踪)+Java agent+bytebuddy 探针》

规划中

《架构师实操篇：基于netty 手写 rpc 框架- 参考 dubbo、seata rpc框架》

《架构师实操篇：go语言学习，以及基于 go 手写 rpc 框架》

《架构师实操篇：千万级任务调度平台 架构与实操- 基于尼恩17年的亿级搜索项目》

《架构师实操篇：工业级 亿级文档搜索 平台 架构与实操- 基于尼恩17年的亿级搜索项目》

特色

会员制

提供技术方向指导，
职业生涯指导，少躺坑，少弯路

简历指导

这个很重要，
对于挪窝涨薪来说

实操性

以上项目，都是老架构师
在生产上实操过的项目

非水货

40岁老架构师，不是水货架构师
《Java高并发三部曲》为证

架构班（社群 VIP）的起源：

最初的视频，主要是给读者加餐。很多的读者，需要一些高质量的实操、理论视频，所以，我就围绕书，和底层，做了几个实操、理论视频，然后效果还不错，后面就做成迭代模式了。

架构班（社群 VIP）的功能：

提供高质量实操项目整刀真枪的架构指导、快速提升大家的：

- 开发水平
- 设计水平
- 架构水平

弥补业务中 CRUD 开发短板，帮助大家尽早脱离具备 3 高能力，掌握：

- 高性能
- 高并发
- 高可用

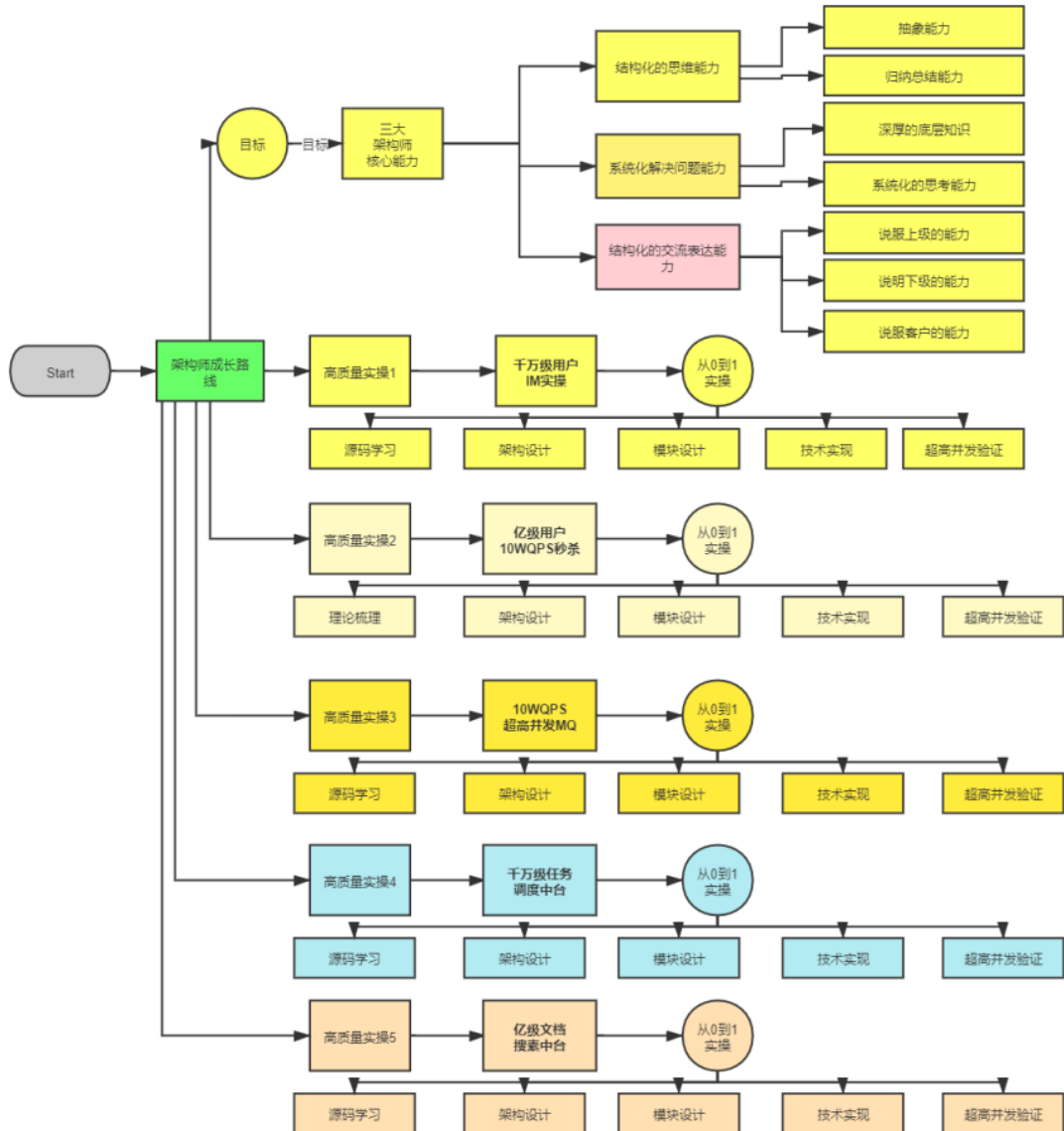
作为一个高质量的架构师成长、人脉社群，把所有的卷王聚焦起来，一起卷：

- 卷高并发实操
- 卷底层原理
- 卷架构理论、架构哲学
- 最终成为顶级架构师，实现人生理想，走向人生巅峰

架构班（社群 VIP）的目的：

- 高质量的实操，大大提升简历的含金量，吸引力，增强面试的召唤率
- 为大家提供九阳真经、葵花宝典，快速提升水平
- 进大厂、拿高薪
- 一路陪伴，提供助学视频和指导，辅导大家成为架构师
- 自学为主，和其他卷王一起，卷高并发实操，卷底层原理、卷大厂面试题，争取狠卷 3 月成高手，狠卷 3 年成为顶级架构师

N 个超高并发实操项目：简历压轴、个顶个精彩



【样章】第 17 章:横扫全网Rocketmq 视频第 2 部曲: 工业级 rocketmq 高可用(HA) 底层原理和实操

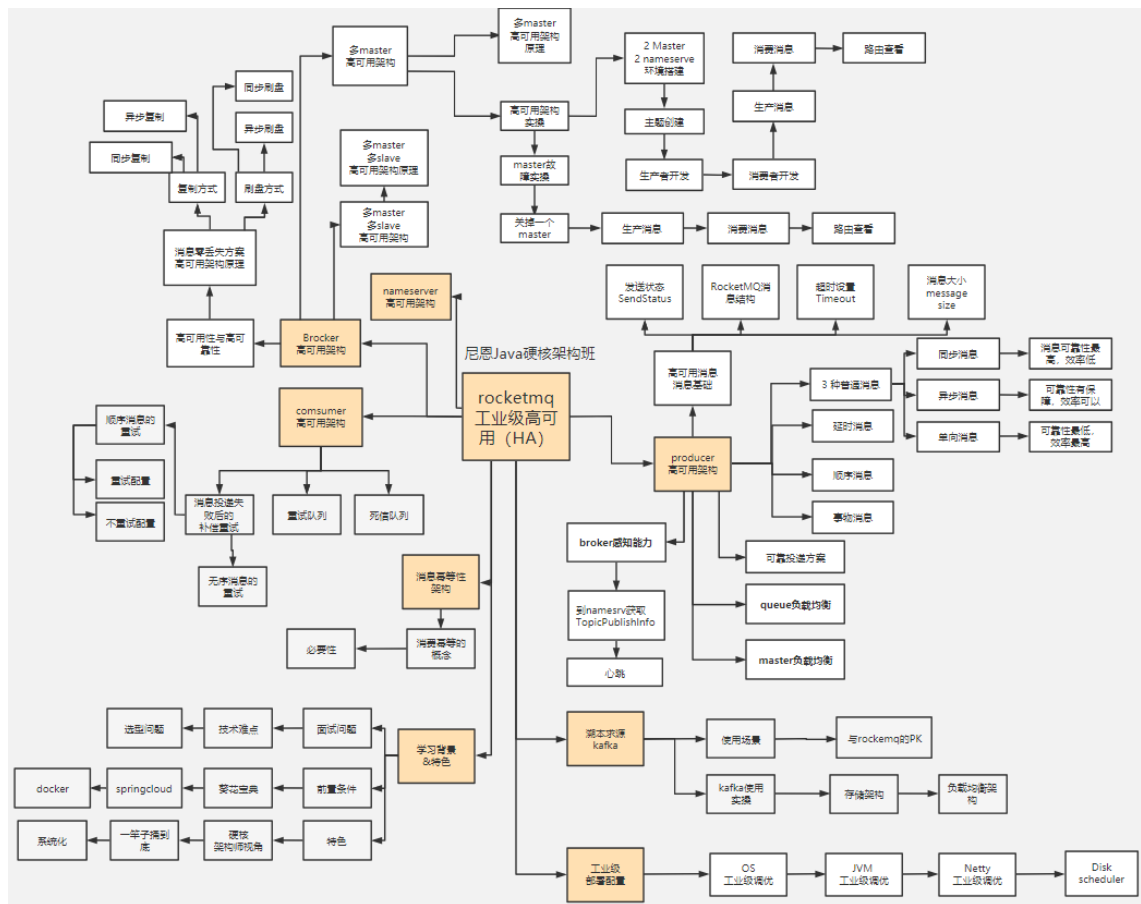
工业级 rocketmq 高可用底层原理, 包含: 消息消费、同步消息、异步消息、单向消息等不同消息的底层原理和源码实现; 消息队列非常底层的主从复制、高可用、同步刷盘、异步刷盘等底层原理。

工业级 rocketmq 高可用底层原理和搭建实操, 包含: 高可用集群的搭建。

解决以下难题:

- 1、技术难题: RocketMQ 如何最大限度的保证消息不丢失的呢? RocketMQ 消息如何做到高可靠投递?
- 2、技术难题: 基于消息的分布式事务, 核心原理不理解
- 3、选型难题: kafka or rocketmq , 该娶谁?

下图链接: <https://www.proceson.com/view/6178e8ae0e3e7416bde9da19>



成功案例：2 年翻 3 倍，35 岁卷王成功转型为架构师

详情: <http://topcoder.cloud/forum.php?mod=forumdisplay&fid=43&page=1>

最新 最后发表 热门 精华

成功案例: [1057号卷王] 3年小伙拿到外企offer, 薪酬涨了200%

1 卷王1号 超级版主 前天 17:41

成功案例: [645号卷王] 4年经验卷王逆袭, 被毕业后, 反涨24W

1 卷王1号 超级版主 2022-9-21

成功案例: [878号卷王] 小伙8年经验, 年薪60W

1 卷王1号 超级版主 2022-8-13

年薪70W案例: 通过尼恩的指导, 小伙伴年薪从40W涨到70W

1 卷王1号 超级版主 2022-2-11

成功案例: [493号卷王] 5年小伙拿满意offer, 就业寒冬季逆涨30%

1 卷王1号 超级版主 前天 17:43

成功案例: [250号卷王] 就业极寒时代, 收offer 涨25%

1 卷王1号 超级版主 前天 17:38

成功案例: [612号卷王] 就业极寒时代, 从外包到自研

1 卷王1号 超级版主 前天 17:15

成功案例: [913号卷王] 热烈祝贺6年经验卷王, 年薪40W

1 卷王1号 超级版主 2022-9-21

成功案例: [959号卷王] 4年经验卷王, 喜获百度、Boss直聘等N个优质offer, 最高涨100%

1 卷王1号 超级版主 2022-9-21

成功案例: [529号卷王] 5年经验卷王喜收2大offer, 最高涨5K

1 卷王1号 超级版主 2022-9-21

成功案例: [811号卷王] 热烈祝贺7年经验卷王, 薪酬涨30%

1 卷王1号 超级版主 2022-9-21

成功案例: [287号卷王] 不惧大寒潮, 卷王逆市收4 offer, 涨30%, 可喜可贺

1 卷王1号 超级版主 2022-5-30

成功案例: [1002号卷王] 5月份“被毕业”, 改简历后, 斩获顶级央企Offer, 涨薪7000+

1 卷王1号 超级版主 2022-7-5

成功案例：[7号卷王] 热烈祝贺小伙伴涨薪120%

① 卷王1号 超级版主 2022-8-13

成功案例：[134号卷王] 大三小伙卷1年，斩获顶级央企Offer，成功逆袭

① 卷王1号 超级版主 2022-7-6

成功案例：[1008号卷王] 5年经验卷王收42W offer，月涨8000，可喜可贺

① 卷王1号 超级版主 2022-5-30

成功案例：[453号卷王] 非全日制 6年卷王喜提3 offer，年薪30W，可喜可贺

① 卷王1号 超级版主 2022-5-21

成功案例：[924号卷王] 6年卷王喜提4 offer，最高涨薪9000，可喜可贺

① 卷王1号 超级版主 2022-5-21

成功案例：[15号卷王] 4年卷王入职 微软，涨薪50%，可喜可贺

① 卷王1号 超级版主 2022-5-12

成功案例：[527号卷王] 4年卷王喜提2 offer，涨薪50%，可喜可贺

① 卷王1号 超级版主 2022-5-13

成功案例：[788号卷王] 3年卷王喜提优质Offer，涨薪60%

① 卷王1号 超级版主 2022-5-11

成功案例：热烈祝贺：非全日制卷王，喜提2个心仪offer，面3家过2家

① 卷王1号 超级版主 2022-4-21

成功案例：[693号卷王] 二线城市6年卷王喜提4大优质Offer，含央企offer，最高薪酬35W

① 卷王1号 超级版主 2022-4-16

成功案例：[85号卷王] 双非2本小伙，春招大捷，喜提9个offer，最高薪酬近30万

① 卷王1号 超级版主 2022-4-14

成功案例：[741号卷王] 卷王逆袭！6年小伙从很少面试机会到搞定35K*14薪Offer

① 卷王1号 超级版主 2022-4-12

成功案例：[642号卷王] 热烈祝贺，6年卷王喜提优质国企offer

① 卷王1号 超级版主 2022-4-7

成功案例：[796号卷王] 热烈祝贺，36岁卷王喜提52万优质offer

① 卷王1号 超级版主 2022-3-25

成功案例: [15号卷王] 小伙卷1年, 涨薪9K+, 喜收ebay等多个优质offer

1 卷王1号 超级版主 2022-3-24

成功案例: [821号卷王] 小伙狠卷3个月, 喜提10多个offer

1 卷王1号 超级版主 2022-3-21

成功案例: [736号卷王] 3年半经验收22k offer, 但是小伙志存高远, 冲击25k+

1 卷王1号 超级版主 2022-3-20

成功案例: 热烈祝贺1群小卷王offer拿到手软, 甚至拒了阿里offer

1 卷王1号 超级版主 2022-3-16

简历案例: 简历一改, 腾讯的邀请就来了! 热烈祝贺, 小伙收到一大堆面试邀请

1 卷王1号 超级版主 2022-3-10

成功案例: 祝贺我国两大超级卷王, 一个过了阿里HR面, 一个过了阿里2面

1 卷王1号 超级版主 2022-3-10

成功案例: 小伙伴php转Java, 卷1.5年Java, 涨薪50%, 喜收多个优质offer

1 卷王1号 超级版主 2022-3-10

成功案例: 4年小伙狠卷半年, 拿到 移动、京东 两大顶级offer

 尼恩 超级版主 2022-3-5

成功案例: [267号卷王] 助力3年经验卷王, 拿到蜂巢的17k x 14薪的offer

1 卷王1号 超级版主 2022-2-27

成功案例: [143号卷王] 二本院校00后卷神, 毕业没到一年跳到字节, 年薪45W

1 卷王1号 超级版主 2022-2-27

成功案例: [494号卷王] 尼恩分布式事务助力卷王拿到 中信银行offer

1 卷王1号 超级版主 2022-2-27

成功案例: [76号卷王] 2线城市卷王, 狠卷1.5年, 喜收22K offer

1 卷王1号 超级版主 2022-2-27

成功案例: [429号卷王] 小伙伴在社群卷5个月, 涨8k+

1 卷王1号 超级版主 2022-2-27

成功案例: [154号卷王] 双非学校毕业卷王, 连拿 京东到家&滴滴 两个大厂Offer

1 卷王1号 超级版主 2022-2-27

☐ 成功案例: [232号卷王] 涨薪10K, 继续卷向食物链顶端

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: 狠卷1年技术, 喜收 腾讯、阿里、微软三大Offer, 最高年薪56W

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: [449号卷王] 应届毕业生卷王喜收 滴滴offer, 年薪33W

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: [551号卷王] 小伙伴学完后, 成功进入大厂, 并且推荐自己的朋友加VIP学习

① 卷王1号 超级版主 2022-2-10

☐ 成功案例: [214号卷王] 助力2年经验卷王, 成功拿到17K月薪

① 卷王1号 超级版主 2022-2-10

☐ 成功案例: [92号卷王] 课程实操助力社群小伙伴喜收 喜马拉雅Offer

① 卷王1号 超级版主 2022-2-10

☐ 成功案例: 社群卷王小伙伴成功过了滴滴三面 获滴滴Offer

① 卷王1号 超级版主 2022-2-10

☐ [612号卷王]滴滴小伙伴, 蹲点考察半年, 觉得靠谱后加入 疯狂创客圈

① 卷王1号 超级版主 2022-2-10

☐ 成功案例: [732号卷王] 尼恩助力3年经验卷王收获 京东offer, 年薪35W

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: [558号卷王] 2年经验卷王, 喜收 网易和阿里子公司两个优质offer

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: [569号卷王] 双非应届生卷王, 喜收字节跳动实习offer

① 卷王1号 超级版主 2022-2-25

☐ 成功案例: [420号卷王] 狠卷1年, 卷王涨薪80%, 涨薪12000元!

① 卷王1号 超级版主 2022-2-25

☐ 成功案例: [76号卷王] 通过尼恩1年半的指导, 专科学历小伙伴从0.8K涨到22K

① 卷王1号 超级版主 2022-2-10

简历优化后的成功涨薪案例（VIP 含免费简历优化）

9年小伙伴拿到年薪90W offer

9月11日改简历 11月29日晒offer

秘诀:
简历指导+ 狠狠卷

薪资高 稳

这个是你微调的

主要的工作是啥?

省路号的地方, 需要你再补充一点

提升了自己的能力, 就不用怕

易所 关于数字货币的

也有打盹的时候, 该裁员, 照样一个不少

上面你留着这个就行

年包比 多 19w

这么多

估计有 70 万

那你不是有 90 个 W?

给我 68

就是吗

Java 开发 9 年-修改.docx 34.1 KB

小伙8年经验 年薪60W

7月12日改简历 8月10日晒offer

秘诀:
改简历+ 狠狠卷

涨幅多少呀?

或者, 幅度多少呀?

之前 36*15, 现在这个 39*15

今年行情不太好, 还有一些 offer 基本都是持平, 没降薪的。

OK

这个马上来

刚在搞简历

哈哈

恭喜恭喜

这次找工作, 您的简历写得超到位了。

我这次面试基本都是看的都是咱们课程的 面试 题。

OK

那就上午 11 点左右吧

好的

6年小伙伴 年薪40W

9月6日改简历 9月21日晒offer

秘诀:
简历指导+ 狠狠卷

张这多 20K

今年这行情, 也算可以了

总包多少呀, 让我也围观一下

大概 40w 吧

谢谢恩师的指导和鼓励

是在深圳

深圳的行情尤其难

能有面试电话就不错了

是的

太牛啦

哈哈哈哈哈, 恩师的鼓励指导也很重要

给你前面调整了一下

简历 - Java - 6 年.docx 24.7 KB

简历 - Java - 6 年(1).docx 24.5 KB

5年小伙喜提3个offer 年薪 35个W

5月22日改简历 11月29日晒offer

恩恩老师晚上好, 汇报下那说的 offer 情况, 最近面试收到了三个 offer, 两个是年薪, 一个是跨境电商公司的 offer, 涨幅暂时不到 20%, 通过这三次面试也让我更加清楚自己距离高级开发还有一点点距离, 还要再多卷卷才能突破。

最后结合自己的情况, 先选择去跨境电商的公司再往下

之前是 20K*13.5, 跨境电商这个是 23K* (14-15)

恭喜恭喜

3 个 offer

就业寒冬, 能拿到 3 个 offer, 已经很不错了, 已经是高手了

很多小伙伴, 面试电话一个都没接到, 简历海投 7000 份, 只收到 3 次面试机会, 没有一个机会拿到最终 offer

您这个年薪, 算下来也有 35W 了吧

年终奖部分还要确认下, 大部分人听说只有两个月

这个时间点, 拿到这个水平, 挺不错的啦

持续加油卷哈

嘿嘿! 看看明年自己有没有能力冲击再开

把你视频都吃透

辛苦老师再帮忙指导下哈

秘诀:
简历指导+ 狠狠卷

我搞好了, 回头找你哈

1.5年小伙搞定15K offer

就业寒冬涨100%

5月7日改简历 11月21日晒offer

秘诀:
简历指导+ 狠狠卷

他那个不止再修修嘛，我才是两位，原先7k，现在15k

那也很牛

都翻倍了，都是牛人

正常就30%

好的

晚上指导你改

推送中台需要加不

我还没做完，准备八脱文新时没时间来搞这些，入职了会开始搞

OK

还有别的项目吗

一个项目，太少啦

那种练习项目也行

其实我工作一年，

那你就更牛逼啦

一年经验，搞定15k offer，舍你其谁

卷王逆袭成功案例

6年小伙从很少面试机会到搞定35K*14薪

3月5日改简历 4月11日拿offer

一个月拿到了理想的offer

面试邀请少 小伙很苦恼

面试法宝 rocketmq四部曲

理想很丰满 目标35K

6年 经验小伙伴

喜收25K offer

3月12日改简历 12月1日晒offer

秘诀:
简历指导+ 狠狠卷

咱们以这个项目为模板改哈

项目有多少人，你带领多少人?

你工作几年了?

6年了

改简历那晚，20 涨到 25 k，高级开发，P7 带的后端团队

任务重，道路远，并不是没有方向

7年经验卷王

薪酬涨30%

7月11日改简历 9月1日晒offer

秘诀:
改简历+ 狠狠卷

只要本事好，拿 offer 比较容易的

入职了，最终并不输大牛嘛，还差旧的大牛，只涨了30%

恭喜一下

现在不比往年

能涨30%是大牛

拿到offer 都算大牛

现在很多人发几百发，这个电话都没有

你已经很牛啦

4年经验卷王逆袭 被毕业后，反涨24W

7月改简历 8月30日晒offer

秘诀:
改简历 + 狠卷

这就是你的简历
差得太多啦
老哥 我八月十号开始找工作，今天已入职了
能涨24W
原因大概是啥?
很多小伙伴，面试机会都没有
这个地方的要这样写
项目被终止
方便语音沟通不

尼恩 你这么指导，非常重要
老哥 我八月十号开始找工作，今天已入职了
现金基本持平，股票 +24W
总计涨了多少钱
股票这个吧只能到手了才算
也不错啦
继续跟着你学技术

小伙5月份"被毕业"，改简历后 斩获顶级央企Offer 涨薪7000+

5月29日改简历 7月5日晒offer

秘诀:
简历指导 + 狠卷3高

快速看书，就能不求甚解，把自读和场景大概一下，然后重点的地方，用到的地方，再去回炉
我被"毕业"了
这周末或下周找你改一下简历
毕业没有关系
ok，发我吧
R行业，就来说去，太复杂啦
嗯，其实有点心理准备
不太会写简历
我拿到手写的offer了
涨20%，没那么多。结果人家都
看起来里面不差钱呀
超过了8000没
平均算下来
7000多
好的
有啥面试的心得吗
可以分享给其他小伙伴的

卷王逆袭成功案例 武汉6年喜收4个优质offer 最高的年薪35W

2月9日改简历 4月15日晒offer

面试法宝:
改简历 + 实操

尼恩老师，新年好!
能帮忙修改下简历吗?
金三银四准备啦
java开发-6年-简历-
拜托了，尼恩，希望能拿25K回来给你报喜
好，我加一下
还有吗?
恩恩，决赛圈offer了
截图是我自认能写出来的评分了。麻烦帮我参考下
选择大于努力，尼恩助我上岸
这么多offer，我看看哈
都是尼恩指点有方，本来还有个新能源汽车的，35W给拒了，主要太远了
跟着尼恩的时间太短了，目前实力也只能到这了
这里边有个大数据的，感觉也不错

卷王逆袭成功案例 6年小伙喜提4个Offer 最高涨9k，年薪35W

4月14日改简历 5月17日晒offer

涨薪法宝:
改简历 + 狠卷

Java开发工程师，
你看着我给你改的
好的呀
谢谢大佬
这个你自己刷
不对的，你你自己刷
那我照着这个改一下库存系统呀
一个简历，
这么漂亮的简历，涨50%，已经没啥问题
只要准备好，不出大错，基本没问题啦
保证押金收起来，你的offer最高涨了9k，多返现100
后面继续跟着你，感觉卷的时间
感觉自己学的不太透彻了
嗯哪
跟着大佬一起
我周围好几个年薪百万的，都是这

卷王逆袭成功案例

非全日制 6年经验卷王

喜提3个Offer，年包30W

5月9日改简历 **5月18日晒offer**

**面试法宝：
改简历 + 狠简历**

聊天记录一：

面试官 VIP453：最新的一个项目的简历还写好吗？
求职者：写好了发你星吗？
面试官 VIP453：那你写好就发我吧。
求职者：嗯嗯好的。
面试官 VIP453：那我先优化下。

聊天记录二：

求职者：高级 Java 开发工程师，三年~6年.docx
面试官 VIP453：恩呢，优化写好了，到时候看看怎么约到啥时候了呢。
求职者：OK，我先看看，然后联系你哈。
面试官 VIP453：估计算几天。
求职者：好的。

聊天记录三：

面试官 VIP453：是啊，有点学到那个这个到问题的思路了。你这个面试回到看面试题试着真的发。
求职者：这个行情看百度看几个博客刷题已经搞不定面试了，太菜了。
面试官 VIP453：真的，现在面试，都要靠运气。
求职者：准备面试点便宜。
面试官 VIP453：不值钱改号。
求职者：对了，我只拿了几个 offer 呀。
面试官 VIP453：回头我打码，删一下吧。
求职者：就是试了4家，搞定了三家。
面试官 VIP453：这个牛逼了。
求职者：哈哈。
面试官 VIP453：OK。
求职者：是啊。
面试官 VIP453：感谢面试官，后面继续努力咯，争取早点拿到架构去。
求职者：再苟3年，差不多就 OK 啦。

卷王逆袭成功案例

4年卷王入职微软，涨50%

3月7日改简历

17:34 周二 ●●● 0.5 MB

< 新简历 Vp15 [redacted] ...

谢谢

3月7日 上午 09:17

超星老师，啥时候有空嘛 麻烦帮我看高简历

👉

**涨薪法宝：
改简历 + 狠狠卷**

java开发工程师简历-[redacted].pdf
478.5 KB

微信电话

推到 10 号了

👉

好嘞，早点找我来找你哈哈哈

去年跟你说的外企我最后没去，还是决定寻找更好的机会

发一份半小时多，很耗时间，一天4小时

OK

5月12日晒offer

17:39 周二 ●●● 0.5 MB

< 新简历 Vp15 [redacted] ...

17:17

超星老师，我入职微软了，在做 azure devops 这块开发，如果群里有想试试微软的我可以给你内推哈哈哈

OK

薪资涨幅如何？

我之前的 14k，现在年薪 28w，虽然薪资不如一统互联网大厂，但是福利还有工作都是挺好的

在无锡

那就好，这个不比大厂差

月薪 20k

还是感谢你的书和面试哈哈哈

也差不多涨了 %50，现在的环境下，非常不错啦

以后我还是会继续跟着看的

4年小伙喜收百度、Boss直聘
等N个顶级Offer
最高涨幅100%

6月27日改简历

9月19日晒offer

秘诀:
改简历+狠狠卷

有个offer选择问题

boss直聘和度小满之间, boss那

还有好几个offer, 还有一个养

总体上是想着boss和度小满之间

了

boss比度小满年包多7w以上,

我这涨幅都快接近百分之百了

卷王逆袭成功案例

4年卷王入收2个offer, 涨50%

3月23日改简历

5月12日晒offer

offer决策

offer决策: n+1 电商erp, 部门成

又搞到个offer

涨薪法宝:
改简历+狠狠卷

涨50%的样子

小伙大三暑期很焦虑
跟着尼恩卷一年
校招斩获顶级央企Offer

去年5月19日加入VIP群

今年7月5日晒offer

秘诀:
狠狠卷书+视频

尼恩老师

我校招去华润电力控股有限公司了

跟着你卷了一年 大学顺便拿了几个国奖

不错不错, 这是央企

这太牛啦

其实拿到手的也就一个a美国一

小伙高中学历
薪酬涨120%

5月6日改简历

7月22日晒offer

你这块估计要涨你668

模块的开发工作, 就这三个哈

哈哈, 不用了老师, 你帮我调了

就行 光今天晚上辅导就值几千了

老师很感谢

还有很多其他的模块

面试官要问

就是其他人做的

嘿嘿, 好

秘诀:
改简历+狠狠卷

5年卷王喜收2大Offer
最高涨5K

5月19日改简历 9月13日晒offer

秘诀:
改简历 + 狠狠卷

发我吧

辛苦了

OK, 简历还需好好改进哈

找主要介绍逻辑, 方法

嗯嗯, 我先消化一下。有问题在请教您

OK

12:20

您好, 我跳槽成功了, 目前手上有两家相对心仪的公司, 想跟您参考下哦

恭喜恭喜🎉

一家是互联网, 主做广告相关的, 给了23K, 一家是自媒体, 做短视频运营的, 给22K

自研的金融科技说

薪资涨幅都不多, 涨幅都不高, 都没达到你涨幅标准🤔

领适

好的, 稍等哈

📎 简历-简历 202205.docx
48.0 KB

微信电脑端

辛苦您了

目前薪资是18K, 目标想冲击到24K

OK, 我晚点看看回复哈

好的感谢

5月21日 周二 20:26

您好, 今天有空看一看我的简历嘛? 帮做优化优化

后面跟着我在腾讯一道

卷王逆袭成功案例
双非二本小伙春招大翻身
喜提9大offer

2月22日改简历 **4月13日晒offer**

WhatsApp Chat 1 (Feb 22):
 h VIP 85 面试官: 您好, 能不能看看有看的后端校招简历?
 java 后端, 无相关项目经验, 期望薪资 14.8k
 OK, 稍晚一点发
 好的谢谢老师
 请问您看有时间发了, 我给您发一下
 没有的
 稍晚一点发, 下午一起发下
 好!

WhatsApp Chat 2 (Apr 13):
 h VIP 85 面试官: 面试官您好
 我研究一下
面试法宝: 改简历+IM实操
 您好, 您发我下下校招的面试资料
 还好好是看校招发了一下面试的还有看您发的简历格式, im和cogs
 谢谢
 面试您的问题基本都能这样问
 谢谢您呀
 17:43

公司	部门	岗位	薪资结构	总包
1. 美团	能源数字化产品部	java后端开发	18.5k+14.5k+5k+200/月+500股票/月	<30w
2. 贝壳	交易研发部	后端	16k+14k	22.4w
3. 贝壳	交易研发部	java游戏开发	15k+5k+餐补+100/月/每月	>22w
4. 字节跳动	服务端	全栈	13k+13k	14.3w
5. 字节跳动	服务端	后端	14k	>16.8w
6. 字节跳动	服务端	后端	9k+包餐补+租房津贴	
7. 字节跳动	服务端	后端	13k+餐补+餐补	17w
8. 字节跳动	服务端	后端	9k+包餐补+租房津贴	
9. 字节跳动	服务端	后端	13k+餐补+餐补	17w

修改简历找尼恩（资深简历优化专家）

- 如果面试表达不好，尼恩会提供 简历优化指导
- 如果项目没有亮点，尼恩会提供 项目亮点指导
- 如果面试表达不好，尼恩会提供 面试表达指导

作为 40 岁老架构师，尼恩长期承担技术面试官的角色：

- 从业以来，“阅历”无数，对简历有着点石成金、改头换面、脱胎换骨的指导能力。
- 尼恩指导过刚刚就业的小白，也指导过 P8 级的老专家，都指导他们上岸。

如何联系尼恩。尼恩微信，请参考下面的地址：

语雀：<https://www.yuque.com/crazymakercircle/gkkw8s/khigna>

码云：<https://gitee.com/crazymaker/SimpleCrayIM/blob/master/疯狂创客圈总目录.md>